
Projet CASSIOPÉE N°50

Disparités entre l'enseignement universitaire et les compétences requises en IA : Une analyse des écarts entre formations et marché de l'emploi

Vincent CHEN Nathan HUBERT Corentin NICODÈME

02/06/2025

Table des Matières

I	Introduction	4
II	Présentation des enjeux du projet	4
II.1	Contexte général	4
II.2	Continuité avec les travaux de recherches précédents	4
II.2.1	Travaux de l'année 2023 : Premiers fondements méthodologiques	4
II.2.2	Travaux de l'année 2024 : Renforcement des mesures et exploration par mots-clés	5
II.2.3	Apports spécifiques de notre recherche	5
II.2.4	Une avancée vers une analyse sémantique intelligente	5
III	Obtention du corpus	6
III.1	Délimitations de l'étude	6
III.2	<i>Web scraping</i> (extraction des données)	6
III.2.1	Offres d'emploi	6
III.2.1.1	Fonctionnement général	6
III.2.1.2	API GraphQL d'Indeed	6
III.2.1.3	Stratégies de filtrage et robustesse	7
III.2.1.4	Construction des résultats	7
III.2.1.5	Intérêt pour notre projet	7
III.2.1.6	Limites et considérations techniques	7
III.2.1.7	Cas de la Nouvelle-Zélande	8
III.2.2	Cours universitaires	8
IV	Traitement automatique du langage naturel	8
IV.1	Prétraitement des données	9
IV.1.1	Normalisation et nettoyage du texte	9
IV.1.2	Renforcement des mots d'intérêt	9
IV.2	Tokenisation	9

IV.2.1	Chargement et configuration du pipeline	9
IV.2.2	1. Tokenisation	10
IV.2.3	Lemmatisation et stemming	10
IV.3	N-grammes	10
IV.3.1	Principe théorique	10
IV.3.2	Détection de N-grammes avec Gensim	11
IV.3.3	Implémentation utilisée	11
IV.3.4	Avantages	12
IV.3.5	Limites	12
IV.4	Vectorisation des textes	12
IV.4.1	TF-IDF : pondération statistique	13
IV.4.2	Word Embeddings : vecteurs denses sémantiques	13
IV.4.3	Comparaison TF-IDF vs Word Embeddings	14
IV.4.4	Place dans notre étude	14
IV.4.5	Conclusion	14
IV.5	Modèle de l'allocation latente de Dirichlet (LDA)	14
IV.5.1	Objectif du modèle	14
IV.5.2	Terminologie et ensemble des paramètres du modèle.	15
IV.5.3	Description globale du modèle et assumptions du modèle.	15
IV.5.4	Apprentissage du modèle	16
IV.5.5	Influence et interprétation de certains paramètres.	17
IV.5.6	Choix du nombre de thèmes (K)	17
IV.5.7	Application à notre problème et améliorations	18
IV.6	Modèle Bidirectional Encoder Representations from Transformers (BERT)	19
IV.6.1	Principe théorique et apports révolutionnaires	19
IV.6.2	Architecture sous-jacente : Le Transformer	19
IV.6.3	Objectifs d'apprentissage (Pré-entraînement)	20
IV.6.4	Représentation de l'entrée et <i>tokens</i> spéciaux	21
IV.6.5	Le mécanisme d'attention bidirectionnelle	22
IV.6.6	Avantages et Applications de BERT	22
IV.6.7	Limitations de BERT	22
IV.6.8	Conclusion et pertinence pour notre étude	23
V	Méthodes de visualisation	23
V.1	Analyse en composantes principales (ACP)	23
V.1.1	Principe	23
V.1.2	Implémentation et résultats	24
V.1.2.1	BERT	24
V.1.2.1.1	Choix de la dimension	24
V.1.2.1.2	Visualisation	26
V.1.2.2	LDA	27
V.1.2.2.1	Visualisation avec pyLDAvis	27
V.2	Uniform Manifold Approximation and Projection (UMAP)	28
V.2.1	Principe	28
V.2.2	Implémentation et résultats	29
V.3	t-Distributed Stochastic Neighbor Embedding (t-SNE)	31
V.3.1	Principe	31
V.3.2	Implémentation et résultats	32

V.4	Synthèse des méthodes de réduction de dimension	33
VI	Similarité sémantique	33
VI.1	Similarité cosinus	33
VI.2	Divergence de Jensen-Shannon	34
VI.3	Indice de Jaccard	34
VI.4	Utilisation sur nos modèles	35
VI.4.1	BERT	35
VI.4.2	LDA	35
VI.4.3	Récapitulatif	37
VII	Résultats	38
VII.1	Étude préliminaire : analyse fréquentielle	38
VII.2	LDA : Résultats préliminaires	39
VII.2.1	Corpus universitaires	39
VII.2.2	Corpus Offres d'emploi	41
VII.2.3	Analyse des distances de Jensen-Shannon	42
VII.3	LDA + Cosinus	43
VII.4	LDA + Jensen-Shannon	43
VII.4.1	Corpus des cours universitaire	43
VII.4.2	Corpus des offres d'emploi	44
VII.5	BERT + Cosinus	46
VII.6	Tests de comparaison à différentes échelles avec BERT	46
VII.6.1	Cours d'un pays vs. Offres d'emploi d'un même pays	47
VII.6.2	Cours d'un pays vs. Offres d'emploi de tous les pays	47
VII.6.3	Cours de toutes les universités vs. Offres d'emploi d'un pays	48
VII.6.4	Cours d'une université vs. Offres d'emploi du même pays	49
VII.6.5	Cours d'une université vs. Offres d'emploi de tous les pays	49
VII.6.6	Cours de toute les universités vs. Offres d'emploi de tous les pays	50
VIII	Conclusion	51
IX	Annexes	52
IX.1	Annexe 1 : Liste des universités étudiées	52
IX.2	Annexe 3 : Schéma récapitulatif de notre pipeline NLP	53
IX.3	Annexe 3 : <i>Whitelist</i>	54
IX.4	Annexe 4 : Diagramme de Gantt	55

I Introduction

L'intelligence artificielle occupe une place croissante dans les sphères professionnelles et académiques. À mesure que cette technologie s'intègre dans un nombre toujours plus vaste de domaines, il devient essentiel d'assurer une adéquation entre les compétences enseignées dans les universités et les exigences du marché du travail. Toutefois, le terme « intelligence artificielle » est devenu un mot-valise englobant une diversité de métiers allant du **Data Scientist** aux **créateurs d'IA générative**. Cette multiplicité d'applications souligne l'importance d'une formation adaptée aux réalités du secteur.

II Présentation des enjeux du projet

II.1 Contexte général

L'essor rapide des technologies numériques dépasse souvent la capacité des institutions académiques à actualiser leurs cursus en temps réel. Pourtant, les universités ont pour mission de former des diplômés capables de répondre aux attentes du marché du travail, notamment dans des domaines en pleine expansion tels que le *machine learning*, le *prompt engineering* ou encore le *cloud computing*.

Ce projet vise à évaluer dans quelle mesure les formations universitaires en **sciences, technologies, ingénierie et mathématiques (STEM)** restent alignées avec les besoins du marché de l'emploi, en constante évolution. À travers cette étude, nous analysons les écarts et convergences entre les compétences enseignées et celles recherchées par les entreprises. L'objectif final est d'identifier d'éventuels ajustements nécessaires dans les programmes universitaires pour garantir une meilleure préparation des étudiants aux réalités du secteur.

II.2 Continuité avec les travaux de recherches précédents

Ce projet Cassiopée s'inscrit dans une dynamique de recherche continue initiée il y a deux ans, visant à analyser l'alignement entre les formations en intelligence artificielle et les compétences attendues dans les offres d'emploi du secteur. À chaque itération annuelle, des méthodes plus poussées ont été introduites, consolidant un pipeline d'analyse basé sur le traitement automatique du langage naturel (NLP), la vectorisation sémantique et la comparaison de similarité.

II.2.1 Travaux de l'année 2023 : Premiers fondements méthodologiques

Le premier groupe de recherche a posé les bases du projet en définissant les objectifs suivants :

- Collecter les contenus pédagogiques de formations (brochures, catalogues de cours) et des offres d'emploi en data science, principalement via **Indeed**.
- Extraire les mots-clés les plus fréquents dans les deux corpus.
- Comparer les thématiques abordées en formation et celles attendues par les employeurs.

Sur le plan technique, ce groupe a :

- Utilisé des scripts Python pour extraire et nettoyer les textes (tokenisation, suppression de ponctuation, mise en minuscules).
- Appliqué un filtrage naïf de *stopwords*, sans lemmatisation.
- Utilisé **TF-IDF** pour pondérer les mots.
- Calculé la **similarité cosinus** entre les vecteurs globaux des formations et des offres.

Les résultats ont montré une première mesure de similarité relativement faible (21 %), mais ont mis en lumière un désalignement important, en particulier autour des compétences pratiques (outils cloud, Python, visualisation) peu présentes dans les syllabus.

II.2.2 Travaux de l'année 2024 : Renforcement des mesures et exploration par mots-clés

L'année suivante, un second groupe a consolidé les fondations en :

- Uniformisant la langue des textes (traduction complète en anglais),
- Nettoyant davantage les corpus (conversion en .txt, standardisation),
- Limitant la dimensionnalité des vecteurs TF-IDF à 500 mots clés.

Leur contribution majeure fut l'introduction d'une **comparaison basée sur les mots-clés extraits**, plutôt qu'une vectorisation brute. Cette méthode, couplée à une **vectorisation CountVectorizer** des mots-clés lemmatisés, a permis d'atteindre une **similarité sémantique de 73 %**, révélant une meilleure correspondance thématique entre les formations et les besoins du marché.

Néanmoins, certaines limites sont restées présentes :

- Approche peu contextuelle (pas de modélisation sémantique profonde),
- Influence forte du choix de vocabulaire et des *stopwords*,
- Modèles statistiques non thématiques (absence de *clustering* sémantique ou de représentation contextuelle).

II.2.3 Apports spécifiques de notre recherche

Notre contribution consiste à franchir un cap méthodologique en intégrant des techniques de **modélisation sémantique avancée** :

- Le **Latent Dirichlet Allocation (LDA)** permet de dégager automatiquement les thématiques dominantes dans les documents.
- Le modèle **BERT** génère des représentations vectorielles contextuelles riches, capables de capturer les relations sémantiques implicites entre les phrases et documents.

Contrairement aux travaux de recherche précédents, qui se concentraient sur la France, nous avons décidé de nous ouvrir aux universités et offres d'emploi **internationales**. Cela va nous permettre d'avoir un corpus beaucoup plus hétérogène, notamment lié à la diversité des universités. Il sera, en plus de l'aspect technique, très intéressant de comparer les universités entre elles, ainsi que les offres d'emploi par pays.

Sur le plan du prétraitement, nous avons :

- Utilisé **spaCy** pour effectuer une lemmatisation robuste et efficace,
- Élaboré une stratégie fine de suppression de **stopwords** personnalisée,
- Introduit la détection automatique de **N-grammes** pour préserver les expressions clés (ex. "**machine learning**", "**cloud computing**").

Enfin, notre architecture permet :

- Une **visualisation interactive des thématiques extraites** (via pyLDAvis),
- Une **évaluation fine de la similarité inter-corpus** basée sur les embeddings BERT,
- Une modularité du pipeline pour adapter l'analyse à d'autres domaines (finance, cybersécurité, etc.).

II.2.4 Une avancée vers une analyse sémantique intelligente

En synthèse, notre travail capitalise sur les acquis des groupes précédents tout en apportant une **modélisation plus fine du langage naturel**. Là où les premières versions reposaient sur des comptages de mots, nous intégrons une **compréhension contextuelle des textes** grâce aux modèles probabilistes et neuronaux récents. Ce tournant méthodologique permet de mieux capturer les écarts sémantiques réels entre les formations et les besoins du marché, ouvrant la

voie à une analyse plus fiable et automatisée de l'adéquation entre l'offre de formation et la demande professionnelle.

III Obtention du corpus

III.1 Délimitations de l'étude

Afin d'assurer une analyse cohérente et pertinente, nous avons choisi de concentrer notre recherche sur huit pays anglophones : le **Royaume-Uni**, les **États-Unis**, le **Canada**, l'**Irlande**, l'**Australie**, la **Nouvelle-Zélande**, l'**Afrique du Sud** et l'**Inde**.

Pour chacun de ces pays, nous avons sélectionné cinq universités reconnues pour l'excellence de leur programme en *Computer Science*. Nous nous sommes exclusivement focalisés sur les formations **undergraduate**, incluant les programmes de Bachelor, afin de couvrir l'ensemble des enseignements proposés dans ce domaine.

Pour plus de détails sur les universités sélectionnées, veuillez consulter l'**Annexe 1 IX.1**.

III.2 *Web scraping* (extraction des données)

III.2.1 Offres d'emploi

Nous avons utilisé le module open source `jobspy`¹, une bibliothèque Python conçue pour extraire des offres d'emploi depuis plusieurs plateformes majeures comme **Indeed**, **LinkedIn**, **Glassdoor** ou **Google Jobs**. Grâce à une interface simple et modulaire, il permet de collecter automatiquement les annonces en fonction de critères précis (pays, mots-clés, date de publication, type de contrat, télétravail, etc.), et d'agrégier les résultats dans un tableau de données standardisé. Dans notre cas, nous avons ciblé la plateforme **Indeed**, particulièrement fiable pour un *scraping* massif sans restrictions.

III.2.1.1 Fonctionnement général

Contrairement à une simple navigation web automatisée, JobSpy interagit directement avec les APIs publiques ou non documentées des plateformes, comme l'API GraphQL d'Indeed. Cela lui permet d'obtenir des données structurées en grande quantité, tout en limitant les risques de blocage. À chaque recherche, une requête est construite dynamiquement selon les filtres définis (intitulé de poste, zone géographique, âge de l'annonce, etc.), puis envoyée à l'API. Les réponses sont ensuite transformées en objets Python standardisés, nettoyés et enrichis avec des informations sur le poste, l'entreprise, la rémunération et le format de contrat.

III.2.1.2 API GraphQL d'Indeed

Contrairement à un *scraping* classique qui repose sur l'analyse du HTML, JobSpy interroge directement l'API GraphQL privée d'Indeed. GraphQL permet de formuler une requête unique pour demander uniquement les champs nécessaires (titre, description, entreprise, salaire, etc.).

L'URL cible est `https://apis.indeed.com/graphql`. À chaque recherche, JobSpy construit une requête contenant :

- un mot-clé (`search_term`)
- une localisation et un rayon (`location`)
- un curseur pour la pagination
- des filtres encodés en fonction des contraintes d'Indeed

¹<https://github.com/speedyapply/JobSpy/tree/main>

L'API répond avec un JSON structuré contenant :

- les résultats (sous `data.jobSearch.results`)
- un curseur pour la page suivante (sous `pageInfo.nextCursor`)

Ce fonctionnement offre :

- un accès plus rapide et fiable aux données
- une meilleure robustesse face aux changements d'interface utilisateur
- une compatibilité directe avec des traitements analytiques en Python

::warning Attention : l'API n'est pas officiellement documentée, et sa stabilité n'est pas garantie. Une mise à jour côté Indeed peut casser le système sans préavis.

III.2.1.3 Stratégies de filtrage et robustesse

JobSpy intègre un moteur de filtrage avancé permettant de combiner des critères complexes, comme la recherche de stages en télétravail publiés depuis moins de 72 heures. Ces filtres sont encodés dans les requêtes envoyées à l'API, en respectant les contraintes spécifiques d'Indeed (exclusion de certains filtres combinés). En cas de panne ou de réponse invalide, JobSpy applique des mécanismes de reprise : délais entre les requêtes, détection des erreurs HTTP, relances conditionnelles et journalisation des incidents.

III.2.1.4 Construction des résultats

Les données reçues depuis l'API sont traitées pour extraire les éléments pertinents : intitulé du poste, description, nom et secteur de l'entreprise, adresse, niveau d'expérience, rémunération (avec intervalle, unité et source), format de contrat, lien direct vers l'annonce, emails détectés, etc. Une attention particulière est portée à la détection des doublons via une base d'URLs déjà vues, et à l'identification du télétravail par analyse croisée des mots-clés dans la description, les attributs et la localisation.

III.2.1.5 Intérêt pour notre projet

Grâce à sa capacité à normaliser et structurer les offres issues de sources hétérogènes, JobSpy constitue un socle idéal pour des analyses textuelles comparatives. En combinant les annonces collectées avec notre pipeline sémantique, nous avons pu étudier les tendances de compétences recherchées, les différences entre pays ou encore la proportion de postes ouverts au télétravail, avec une granularité fine.

III.2.1.6 Limites et considérations techniques

Certaines limitations méritent d'être prises en compte pour une utilisation à grande échelle :

- La plupart des plateformes imposent une **limite de résultats récupérables**, souvent autour de **1000 offres** par requête, même via l'API. Cela implique de bien définir ses filtres de recherche pour éviter les résultats tronqués ou trop génériques.
- Les **politiques anti-bot** sont particulièrement strictes sur des sites comme LinkedIn. Bien que JobSpy permette l'utilisation de **proxies** pour contourner les blocages (via rotation d'adresses IP), cela nécessite une infrastructure réseau plus complexe en cas d'usage massif. Dans notre cas, nous avons été incapable de récupérer 1000 offres d'emploi via LinkedIn.
- En fonction des plateformes, certaines **données sont partielles** ou manquantes, et leur structure peut varier d'un pays à l'autre. Cela confirme une nouvelle fois l'importance à porter sur la phase de nettoyage des données.
- Enfin, comme l'API d'Indeed utilisée par JobSpy **n'est pas officielle**, elle peut être modifiée ou désactivée sans préavis. Nous avons pu récupérer notre base de données mais nous ne

sommes pas assurés de pouvoir la mettre à jour (notamment avec des offres d'emploi plus récentes) dans le futur, en utilisant ce module.

III.2.1.7 Cas de la Nouvelle-Zélande

Nous avons constaté que nous n'avons obtenu que 223 offres d'emploi pour la Nouvelle-Zélande, au lieu des 1000 initialement souhaitées. Malgré plusieurs ajustements des périodes de recherche, nous n'avons pas réussi à obtenir davantage de données. Afin de maintenir une méthode de *scraping* cohérente, nous avons choisi de ne pas extraire des données à partir d'autres sites, afin d'éviter tout biais lié à la source des informations. Nous tiendrons compte de cette différence de volume de données dans l'interprétation des résultats.

III.2.2 Cours universitaires

Le *scraping* des sites universitaires s'est avéré particulièrement complexe en raison de la grande hétérogénéité des structures web et de l'absence de standard dans la présentation des programmes. Contrairement aux offres d'emploi, les descriptifs de cours (*syllabi*, *course outlines*) sont rarement disponibles sous forme structurée ou centralisée.

Pour contourner cette difficulté, nous avons procédé à une recherche manuelle des pages pertinentes, en ciblant les *syllabi* accessibles publiquement. Ce travail s'est révélé extrêmement chronophage, certaines universités exigeant des identifiants pour consulter leurs contenus pédagogiques.

Dans la majorité des cas, chaque cours dispose de sa propre page descriptive, ce qui implique une consultation individuelle et fragmentée. Afin de constituer un corpus exploitable, nous avons copié manuellement les contenus textuels dans des fichiers bruts. Cette méthode, bien que simple, engendre plusieurs problèmes :

- **Présence de bruit textuel** : titres génériques, coordonnées, mentions administratives redondantes (ex. : « Course outline », « Coordinator email », etc.)
- **Éléments inutiles à l'analyse** : numérotations, dates, codes de cours, ponctuation répétitive
- **Structure non homogène** : tabulations, indentations et mises en page variables selon les sites

Ces éléments sont répétés dans chaque fiche de cours et constituent un **bruit systématique** dans les données. Une phase de **nettoyage approfondi** est donc indispensable avant toute application de modèles de traitement automatique du langage (NLP).

IV Traitement automatique du langage naturel

À partir des données obtenues, nous réalisons une étape de prétraitement afin de supprimer les résultats aberrants dus au langage. Ensuite, il faut traiter les mots restants selon une méthode NLP (*Natural Language Processing*), nous avons initialement considéré l'utilisation de LDA (*Latent Dirichlet Allocation*), mais nous verrons que nous pouvons utiliser d'autres modèles.

Afin de faciliter la compréhension de notre pipeline NLP, vous pourrez trouver en Annexe 2 IX.2 un schéma récapitulant les différentes étapes de traitement.

IV.1 Prétraitement des données

IV.1.1 Normalisation et nettoyage du texte

L'un des principaux défis de cette analyse repose sur la non-normalisation des données textuelles extraites, qui contiennent des éléments non pertinents pour l'analyse sémantique. Par exemple, lors du *scraping*, des caractères parasites apparaissent, tels que des espaces, des numérotations, des tirets, etc. Avant tout traitement, nous devons donc systématiquement filtrer et structurer ces données.

1. **Suppression des *stopwords*** : Certains mots récurrents, comme “the”, “is”, “a” en anglais, n'apportent que peu de valeur sémantique. Ces *stopwords* sont systématiquement retirés pour éviter d'introduire du bruit dans l'analyse.

Pour obtenir cela, nous avons utilisé le lexique (liste de mots) *stopwords* du package python `nltk.corpus`², auquel nous avons rajouté manuellement certains termes sans intérêt que nous avons fréquemment rencontrés.

2. **Suppression des caractères spéciaux et des ponctuations** : Certains caractères n'apportent pas d'information linguistique exploitable et sont donc retirés. Nous avons pour cela utilisé les **expressions régulières** (regex), implémentées grâce au module `re` en python³.
3. **Uniformisation du texte** : Toutes les lettres sont converties en minuscules afin d'éviter que des variations typographiques ne soient considérées comme des entités distinctes.

IV.1.2 Renforcement des mots d'intérêt

Pour améliorer l'entraînement de nos modèles, nous avons accentué la fréquence d'apparition des mots appartenant au champ lexical des *computer sciences*. Pour identifier ces mots, nous avons créé une *whitelist* que vous pouvez trouver dans l'**Annexe 3 IX.3**. Le but est de multiplier par un facteur $k \in [3, 10]$ tous les termes du corpus présents dans cette liste. Nous verrons par la suite que notre traitement utilise la fréquence d'apparition des termes pour relever les topics les plus importants. Il est alors facile de comprendre l'importance de cette étape.

Cependant, pour certains modèles de *deep learning* (en l'occurrence BERT dans notre étude), leur fonctionnement ne nous permet pas d'utiliser cette *whitelist* car ils considèrent chaque mot dans leur ensemble (par rapport à une phrase par exemple), et non pas individuellement. La duplication du terme entraîne donc une perte de sens dans la phrase et est donc proscrite.

IV.2 Tokenisation

Pour commencer le traitement automatique du langage naturel, nous avons utilisé la bibliothèque Python `spaCy`⁴, reconnue pour sa rapidité et sa précision dans les tâches de prétraitement linguistique. Elle est conçue autour d'une architecture modulaire et hautement optimisée, reposant sur des modèles statistiques et des règles linguistiques spécifiques à chaque langue.

IV.2.1 Chargement et configuration du pipeline

Nous avons utilisé le modèle pré-entraîné léger `en_core_web_sm`, adapté aux ressources standards. Le chargement se fait via :

²<https://www.nltk.org/howto/corpus.html>

³<https://docs.python.org/fr/3.13/library/re.html>

⁴<https://spacy.io/models/en>

```
spacy.cli.download("en_core_web_sm")
nlp = spacy.load("en_core_web_sm", disable=["parser", "ner"])
```

Le pipeline de traitement de **spaCy** se compose de plusieurs modules, dont :

- **Tokenizer** : segmentation du texte en unités lexicales (*tokens*)
- **Tagger** : assignation de catégories grammaticales (**POS tagging**)
- **Lemmatizer** : réduction des mots à leur *lemme*
- **Parser** : analyse syntaxique (désactivée ici)
- **NER** : reconnaissance d'entités nommées (désactivée ici)

La désactivation des modules **Parser** et **NER** permet de réduire significativement le temps de traitement, car ils reposent sur des réseaux de neurones complexes (par exemple, **BiLSTM** pour la dépendance syntaxique).

IV.2.2 1. Tokenisation

La tokenisation dans **spaCy** repose sur un moteur déterministe basé sur des expressions régulières combinées à des règles spécifiques à la langue. Le processus permet de segmenter le texte brut T en une suite ordonnée de *tokens* $T = (t_1, t_2, \dots, t_n)$.

Contrairement aux approches naïves (ex. `split` sur les espaces en python), **spaCy** gère finement :

- la ponctuation (ex : « U.S. » $\neq$ « U » + « . » + « S » + « . »),
- les contractions (ex : « don't » $\rightarrow$ « do » + « not »),
- les entités agglomérées (ex : dates, montants, e-mails...).

IV.2.3 Lemmatisation et stemming

L'intérêt de ce traitement est de ne pas distinguer dans notre corpus deux mots qui sont sémantiquement identiques. Nous appliquons pour cela des techniques de réduction morphologique :

- **Stemming** : Réduction des mots à leur racine (ex : `computing` $\rightarrow$ `comput`).
- **Lemmatisation** : Réduction des mots à leur forme canonique (ou *lemme*) en tenant compte de la grammaire (ex : `running` $\rightarrow$ `run`).

Pour cela, on utilise une combinaison :

1. de dictionnaires linguistiques (*WordNet-like*),
2. de règles morphologiques basées sur les suffixes et préfixes,
3. du contexte grammatical (issu du **POS tagger**) pour choisir entre plusieurs *lemmes* possibles.

Formellement, soit un token t_i , son *lemme* est défini comme :

$$\text{lemme}(t_i) = \arg \min_{\{l \in \mathcal{L}\}} \text{Cost}(t_i, l \mid \text{POS}(t_i))$$

où $\mathcal{L}$ est l'ensemble des *lemmes* candidats, et le coût **Cost** dépend des règles linguistiques intégrées.

IV.3 N-grammes

IV.3.1 Principe théorique

Un **N-gramme** est une séquence contiguë de n éléments (dans notre cas des mots) extraite d'un texte. Cette approche permet de capturer certaines régularités locales du langage naturel que les modèles « sac de mots » classiques ne prennent pas en compte. On distingue notamment :

- Les **unigrammes** : séquences d'un seul mot.
- Les **bigrammes** : paires de mots consécutifs (ex. « machine learning »).

- Les **trigrammes** : séquences de trois mots consécutifs (ex. « natural language processing »).

L’objectif de cette technique est de capturer les cooccurrences significatives qui reflètent des expressions composées, terminologies techniques ou collocations courantes. Par exemple, les termes “**data science**” ou “**deep learning**” sont plus informatifs que leurs constituants séparés.

IV.3.2 Détection de N-grammes avec Gensim

Pour détecter automatiquement ces expressions, nous avons utilisé le module `gensim.models.Phrases`⁵, qui construit un modèle statistique basé sur la fréquence de cooccurrence des paires de mots.

Le fonctionnement repose sur la mesure d’une **association** entre deux *tokens* w_i et w_{i+1} , évaluée via un **score de PMI (Pointwise mutual information)** modifiée. Cette information mutuelle ponctuelle est une mesure d’association qui compare la probabilité que deux événements se produisent ensemble à ce que serait cette probabilité si les événements étaient indépendants.

Le PMI d’une paire de résultats x et y appartenant à des variables aléatoires discrètes X et Y quantifie l’écart entre la probabilité de leur coïncidence compte tenu de leur distribution conjointe et de leurs distributions individuelles, en supposant qu’elles sont indépendantes :

$$\text{pmi}(x, y) = \log_2 \frac{p(x, y)}{p(x) \cdot p(y)} = \log_2 \frac{p(x|y)}{p(x)} = \log_2 \frac{p(y|x)}{p(y)}$$

Appliqué à la détection de **N-grammes**, on utilise cette intuition pour mesurer la force d’association entre deux mots w_i et w_{i+1} , en comparant la probabilité de leur apparition conjointe à celle attendue si les mots étaient indépendants. Cela permet d’identifier les cooccurrences significatives formant des expressions lexicales stables.

En pratique, une version simplifiée et régularisée de cette mesure est utilisée :

$$\text{score}(w_i, w_{i+1}) = \frac{\text{count}(w_i, w_{i+1}) - \delta}{\text{count}(w_i) \cdot \text{count}(w_{i+1})}$$

où :

- $\text{count}(w_i, w_{i+1})$ est la fréquence observée de la bigramme,
- δ est un seuil de régularisation (fixé empiriquement),
- Le dénominateur correspond à la fréquence attendue si w_i et w_{i+1} étaient distribués de manière indépendante.

Un score élevé indique que les deux mots coapparaissent fréquemment ensemble de manière significative. Les bigrammes dépassant un seuil fixé sont alors fusionnés (ex. « `deep_learning` »).

IV.3.3 Implémentation utilisée

Le code suivant détecte automatiquement les bigrammes et trigrammes dans une collection de documents déjà tokenisés :

```
def ngrams_detection(data: List[List[str]]) -> List[List[str]]:
    bigrams = gensim.models.Phrases(data, min_count=2, threshold=10)
    trigrams = gensim.models.Phrases(bigrams[data], threshold=10)
    bigram_mod = gensim.models.phrases.Phraser(bigrams)
```

⁵<https://radimrehurek.com/gensim/models/phrases.html>

```
trigram_mod = gensim.models.phrases.Phraser(trigrams)
return [trigram_mod[bigram_mod[text]] for text in data]
```

Interprétation du code:

- `min_count=2` : seuls les mots ou paires apparaissant au moins deux fois sont pris en compte pour éviter les associations bruitées.
- `threshold=10` : seuil d'inclusion basé sur la force de l'association. Plus il est élevé, plus seules les cooccurrences fortes seront retenues.
- `Phrases` : construit un modèle de détection de phrases (bigrammes/trigrammes).
- `Phraser` : transforme le modèle en une version optimisée pour des appels rapides (transformation de texte).

L'application successive du modèle de bigrammes puis de trigrammes permet d'obtenir des séquences comme :

```
["deep", "learning", "is", "powerful"] → ["deep_learning", "is", "powerful"]
["natural", "language", "processing"] → ["natural_language_processing"]
```

IV.3.4 Avantages

Voici quelques avantages que nous avons trouvés à son utilisation :

- Réduction de l'ambiguïté sémantique : “*artificial intelligence*” devient une seule unité lexicale.
- Meilleure structuration des corpus pour le *topic modeling* ou les modèles de similarité.
- Préservation des collocations techniques fréquentes dans les offres d'emploi et *syllabi*.

IV.3.5 Limites

Par ailleurs, ce traitement a plusieurs limites que nous avons identifiées :

- Le modèle est basé uniquement sur des statistiques de cooccurrence : il ne tient pas compte du contexte sémantique global.
- L'ordre des transformations (bigrammes avant trigrammes) influe sur le résultat final.
- Les séquences disjointes ($n > 3$ ou non consécutives) ne sont pas détectées.

IV.4 Vectorisation des textes

Après cette phase de nettoyage linguistique, chaque document textuel est transformé en une représentation numérique vectorielle, exploitable par les modèles d'apprentissage automatique. Cette étape est indispensable pour passer du texte brut à une forme mathématiquement traitable.

Deux grandes approches de vectorisation sont utilisées :

- **Bag-of-Words (BoW)** : chaque document est représenté comme un vecteur creux de dimension V (taille du vocabulaire), indiquant la fréquence ou la pondération (ex. TF-IDF) de chaque mot. Cette méthode ignore l'ordre des mots mais reste très efficace pour les modèles statistiques comme LDA (que nous détaillerons plus tard).
- **Word Embeddings** : chaque mot est projeté dans un espace vectoriel dense, où les mots proches sémantiquement ont des vecteurs proches. Ces vecteurs sont appris à partir de grands corpus (Word2Vec, GloVe, FastText).

Dans notre étude, nous nous sommes intéressés aux deux approches.

IV.4.1 TF-IDF : pondération statistique

Le schéma de pondération **TF-IDF** (**T**erm **F**requency – **I**nverse **D**ocument **F**requency) permet de transformer un document textuel en un vecteur numérique pondéré. Cette transformation repose sur une double intuition :

- Un mot est représentatif d'un document s'il y est fréquent (**TF**),
- Mais il devient peu informatif s'il est aussi fréquent dans tout le corpus (**IDF**).

La formule canonique est :

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \log\left(\frac{N}{\text{DF}(t)}\right)$$

où :

- $\text{TF}(t, d)$ est la fréquence relative du terme t dans le document d ,
- $\text{DF}(t)$ est le nombre de documents contenant le terme t ,
- N est le nombre total de documents du corpus.

L'application de TF-IDF permet de faire ressortir les termes fortement représentatifs des documents analysés et de limiter l'impact des mots trop courants.

Cette formule permet d'obtenir un vecteur creux (*sparse*) de dimension V (taille du vocabulaire), où chaque position représente un mot pondéré. Ce type de représentation est particulièrement adapté à :

- La détection de thèmes via LDA (lorsqu'on veut filtrer les termes peu discriminants),
- Les modèles de classification ou de clustering (avec une métrique comme la distance cosinus, que nous aborderons plus tard).

Cependant, TF-IDF souffre d'une limitation importante : il ignore complètement la **sémantique des mots**. Deux mots synonymes (ex. AI et artificial intelligence) auront des vecteurs totalement différents.

IV.4.2 Word Embeddings : vecteurs denses sémantiques

Pour pallier cette limitation, les **word embeddings** sont des représentations **denses** et **faiblement dimensionnelles** (typiquement 100 à 300 dimensions), apprises de manière à refléter les similarités contextuelles des mots.

Le principe général repose sur la distribution linguistique formulée par Firth (1957) : « *You shall know a word by the company it keeps.* »

Les modèles les plus connus incluent :

- **Word2Vec** (Mikolov et al., 2013) :
 - Deux architectures : **CBOW** (Continuous Bag-of-Words) et **Skip-Gram**.
 - Optimise la probabilité d'apparition de mots dans un contexte local.
- **GloVe** (Pennington et al., 2014) :
 - Utilise les cooccurrences globales des mots dans une matrice pondérée.
 - Résout un problème de factorisation matricielle.

Les embeddings permettent de faire apparaître des régularités sémantiques dans l'espace vectoriel :

$$(\text{king}) - (\text{man}) + (\text{woman}) \approx (\text{queen})$$

où (word) est la représentation vectorielle de word

IV.4.3 Comparaison TF-IDF vs Word Embeddings

Critère	TF-IDF	Word Embeddings
Représentation	Sparse	Dense
Sémantique	Absente (comptage brut)	Présente (via contexte)
Dimensions	Taille du vocabulaire	Fixe (100–300 typiquement)
Captation du contexte	Non	Oui (locaux ou globaux)
Utilisation en LDA	Utilisé comme filtre	Non compatible
Utilisation en clustering	Compatible	Compatible
Utilisation en BERT	Inutile (BERT apprend les siennes)	Inutile

IV.4.4 Place dans notre étude

- Pour **LDA**, TF-IDF sert à **filtrer les termes** mais la modélisation se fait sur des fréquences brutes (BoW).
- Pour des **modèles supervisés classiques** (SVM - *Support vector machine*, KNN - *k-nearest neighbors...*), TF-IDF ou embeddings peuvent être utilisés comme vecteurs d'entrée.
- Pour des **modèles de similarité** (cosinus, K-means), que nous expliciterons par la suite, les deux approches sont possibles, avec des performances souvent meilleures pour les embeddings.
- Pour **BERT**, ni TF-IDF ni Word2Vec ne sont nécessaires : le modèle génère lui-même des embeddings contextuels via son architecture transformer.

IV.4.5 Conclusion

La pondération TF-IDF constitue une méthode simple et efficace pour extraire les termes discriminants dans un corpus. Elle est adaptée aux modèles statistiques classiques et aux tâches où l'interprétabilité est primordiale. En revanche, les **word embeddings** offrent une représentation plus riche et sémantique, particulièrement utile pour des modèles profonds ou des analyses de similarité fine. Notre choix dépend donc du type de modèle utilisé et de la nature de la tâche.

IV.5 Modèle de l'allocation latente de Dirichlet (LDA)

IV.5.1 Objectif du modèle

Le principe de la **LDA** ou **Latent Dirichlet Allocation** est étant donné un nombre de thèmes K , de déterminer ceux-ci tout en allouant des probabilités d'appartenance pour chaque mot à ces thèmes. L'algorithme permet donc de catégoriser l'ensemble des mots dans un ensemble de documents en un petit nombre de sujets (ou topics) qui décrivent au mieux le document. L'objectif dans notre cas en particulier est de transformer la représentation des corpus de documents en un ensemble de distributions thématiques, ce qui facilitera leur analyse comparative en se concentrant sur les concepts sous-jacents plutôt que sur les mots individuels.

La LDA permettra d'identifier les thèmes prédominants dans les offres d'emploi (compétences requises) et dans les programmes universitaires (compétences enseignées), offrant ainsi une base sémantique pour la comparaison des écarts.

IV.5.2 Terminologie et ensemble des paramètres du modèle.

Pour commencer, nous avons besoin de définir un peu de terminologie. On appelle **document** un ensemble (au sens mathématique) de mots. Un **thème** (parfois sujet ou topic) est une distribution de probabilité sur l'ensemble des mots. Un **thème** ordonne ainsi l'ensemble des mots du document en fonction de leur probabilité d'appartenance à celui-ci, autrement dit leur importance pour ce thème en particulier.

Paramètre	Signification
K	Nombre de thèmes
M	Nombre de documents
N_i	Nombre de mots pour le $i^{\text{ème}}$ document
V	Ensemble du vocabulaire ou la réunion des $N_k, k \in M$
α	Paramètre de la distribution de Dirichlet utilisée pour obtenir les distributions de probabilité $\theta_i, i \in [1, M]$ caractérisant la répartition des thèmes parmi les M documents Un α élevé encourage des documents à contenir un mélange de nombreux thèmes, tandis qu'un α faible pousse les documents à être dominés par un petit nombre de thèmes
β	Paramètre de la distribution de Dirichlet utilisée pour obtenir les distributions de probabilité $\Phi_k, k \in [1, K]$ sur l'ensemble des mots associées aux K thèmes Un β élevé encourage des thèmes à contenir un mélange de nombreux mots du vocabulaire, tandis qu'un β faible pousse les thèmes à être dominés par un petit nombre de mots très spécifiques
θ_i	Vecteur de taille K donnant une distribution de probabilité sur les K thèmes pour le document i décrivant l'importance de chaque thème pour le document i
Φ_k	Vecteur de taille V donnant la distribution de probabilité sur l'ensemble des mots pour le thème k , décrit l'importance de chaque mot pour caractériser ce thème
w_{ij}	$j^{\text{ème}}$ mot du $i^{\text{ème}}$ document
z_{ij}	Thème associé au mot w_{ij}

Tableau 1. – Ensemble des paramètres de la LDA

IV.5.3 Description globale du modèle et assumptions du modèle.

Le modèle de la LDA nécessite le modèle probabiliste suivant : Les distributions des sujets pour décrire documents mais aussi celles des mots pour décrire chacun des thèmes suivent des lois

de Dirichlet. Les variables latente donnant l'appartenance d'un mot donné à un document ou un thème suit une loi catégorique (multinomiale à un seul tirage). On a ainsi :

$$\forall i \in \llbracket 1, M \rrbracket \theta_i \sim \text{Dir}_K(\alpha)$$

→ Chaque document i a une distribution de topics θ_i tirée d'une Dirichlet avec le paramètre de concentration α , qui est un vecteur de taille K

$$\forall k \in \llbracket 1, K \rrbracket \Phi_k \sim \text{Dir}_V(\beta)$$

→ Chaque topic k a une distribution de mots Φ_k tirée d'une Dirichlet avec le paramètre de concentration β , qui est un vecteur de taille V

$$\forall i \in \llbracket 1, M \rrbracket \forall j \in \llbracket 1, N_i \rrbracket z_{ij} \sim \text{Categorical}(\theta_i)$$

$$\forall i \in \llbracket 1, M \rrbracket \forall j \in \llbracket 1, N_i \rrbracket w_{ij} \sim \text{Categorical}(\varphi_{z_{ij}})$$

• **Processus de Génération des thèmes pour le document i**

1. Choisir θ_i suivant la loi de Dirichlet $\text{Dir}(\alpha)$
2. Choisir Φ_k suivant la loi de Dirichlet $\text{Dir}(\beta)$
3. Pour chaque mot de position j dans le document i :
 - a. Choisir un thème z_{ij} suivant la loi catégorique $\text{Categorical}(\theta_i)$
 - b. Choisir un mot w_{ij} suivant la loi catégorique $\text{Categorical}(\Phi_{z_{ij}})$.

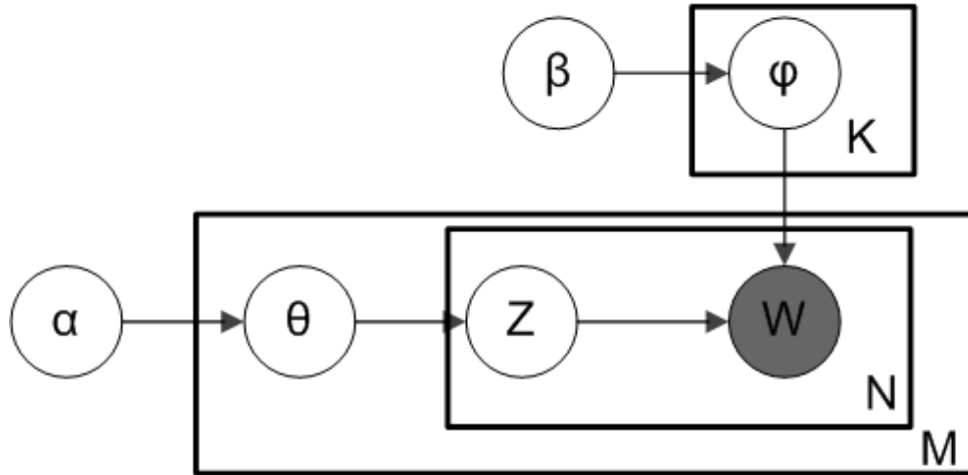


Fig. 1. – Notation de Plate pour la LDA

La notation de Plate ci-dessus permet de représenter visuellement le modèle que nous venons de décrire. Les boîtes représentent les étapes de générations répétées. Les dépendances entre les variables aléatoires sont décrites sous la forme de flèches. La variable W est grisée pour montrer que celle-ci ne peut être qu'observée.

IV.5.4 Apprentissage du modèle

On suppose que les mots w sont générés à partir de distributions catégoriques associées aux documents (d) et aux thèmes (t) :

$$\mathbb{P}(w|d) = \sum_{t=1}^K \mathbb{P}(w|t)\mathbb{P}(t|d)$$

où :

- $\mathbb{P}(w|t)$ est la probabilité que le mot w soit généré par le thème t ,
- $\mathbb{P}(t|d)$ est la probabilité que le thème t décrive le document d .

On commence donc par assigner aléatoirement les mots aux thèmes. Pour améliorer le modèle on réalise récursivement les étapes suivantes :

Pour chacun des mots w de chaque document d , on considère que tous les thèmes assignés sont correct, sauf celui du mot choisi. En utilisant le principe de l'inférence Bayésienne, on calcule pour chaque thème t la probabilité que ce thème soit assigné au mot w_{ij} étant donné les assignations de tous les autres mots et les documents. Le thème t^* qui maximise cette probabilité est alors assigné au mot w_{ij} .

Les assignations finissent par se stabiliser en répétant les étapes précédentes, ce qui nous donne un modèle de LDA pour la distribution des mots pour chaque thème.

L'objectif de l'apprentissage est de trouver les distributions θ et Φ qui maximisent la probabilité des documents observés.

IV.5.5 Influence et interprétation de certains paramètres.

• Influence de α

Comme dit plus tôt, le paramètre α permet de contrôler la répartition des thèmes, c'est à dire de leur importance, au sein de chacun des documents. Autrement dit, α contrôle à quel point la distribution θ_i est concentré sur un faible nombre de thème. Un grand α aura tendance à donner une distribution plus uniforme, Cela signifie qu'un grand nombre de sujets sont importants pour décrire chacun des documents. A l'inverse, un α faible aura tendance à assigner un nombre plus faible de thèmes à chacun des documents, cela donnera ainsi des thèmes plus spécifiques à chacun des documents et plus précis.

α permet ainsi de décrire notre connaissance *a priori* du nombre de sujets qu'un document à tendance à couvrir.

• Influence de β

De façon similaire au paramètre précédent, β contrôle la distribution de probabilités sur l'ensemble du vocabulaire Φ_k permettant de décrire les thèmes. Un grand β aura tendance à rendre les distributions de mots pour chaque thème plus uniformes, signifiant que chaque thème est décrit par un grand nombre de mots, y compris des mots moins discriminants. À l'inverse, un β faible encouragera des thèmes à être définis par un nombre plus restreint de mots très spécifiques et fortement associés à ce thème, rendant les thèmes plus distincts mais potentiellement plus spécialisés.

• Symétrie/asymétrie

Dans la pratique, α et β sont souvent traités comme des scalaires (cas symétrique) où chaque composante du vecteur Dirichlet correspondant à la même valeur. Il est également possible d'utiliser des versions asymétriques pour affiner le modèle si des connaissances *a priori* sur la distribution des thèmes ou des mots sont disponibles.

IV.5.6 Choix du nombre de thèmes (K)

La détermination du nombre optimal de thèmes K est une étape cruciale et souvent délicate dans l'application de la LDA, car il n'existe pas de méthode unique et universellement reconnue pour cette tâche. Un K trop petit peut entraîner la fusion de sujets distincts en un seul thème large

et peu précis, tandis qu'un K trop grand peut générer des thèmes redondants, trop spécifiques ou peu cohérents.

Plusieurs approches peuvent être utilisées pour guider ce choix :

- **Cohérence des thèmes (Coherence Score):** Cette métrique évalue la cohérence sémantique des mots au sein de chaque thème. L'idée est que les mots fortement liés à un même thème devraient apparaître fréquemment ensemble dans le corpus. Des métriques de cohérence courantes incluent :
 - C_V : Considérée comme l'une des plus robustes, C_V est basée sur une combinaison de mesures de segmentation de fenêtres, de la similarité cosinus et de la normalisation. Elle évalue la similarité des paires de mots les plus pertinents pour chaque thème.
 - **UMass**: Plus simple à calculer, UMass évalue la cohérence en fonction de la co-occurrence des mots.

L'approche typique consiste à calculer le score de cohérence pour une plage de valeurs de K (par exemple, de 5 à 50 par pas de 5) et à choisir la valeur de K qui maximise ce score ou qui présente un « coude » (point d'inflexion) significatif sur le graphique des scores.

- **Perplexité:** Traditionnellement utilisée dans les modèles de langage, la perplexité mesure à quel point un modèle de probabilité prédit bien un échantillon. Dans le contexte de la LDA, une perplexité plus faible indique généralement un meilleur modèle. Cependant, la perplexité a tendance à diminuer continuellement avec l'augmentation de K , ce qui la rend moins utile pour identifier un K optimal sans autre critère. Elle est souvent considérée comme un indicateur de la capacité de généralisation du modèle.
- **Interprétation humaine:** Malgré l'existence de métriques automatiques, l'interprétation qualitative des thèmes par des experts du domaine reste essentielle. Après avoir entraîné le modèle pour différentes valeurs de K , il est impératif d'examiner les mots clés associés à chaque thème et de déterminer s'ils sont sémantiquement distincts, significatifs et utiles pour les objectifs de l'analyse. C'est souvent le critère le plus important pour des applications pratiques. Un compromis entre les métriques automatiques et l'interprétation humaine est généralement la meilleure approche.

IV.5.7 Application à notre problème et améliorations

Pour appliquer efficacement le modèle LDA à notre problématique, nous avons mis en place une procédure d'optimisation des hyperparamètres reposant sur une recherche systématique des combinaisons les plus performantes. L'idée centrale était de tester différentes valeurs pour les paramètres clés tels que le nombre de thèmes, les distributions de Dirichlet α et β , le nombre de passes sur le corpus, ainsi que la taille des blocs de traitement. Pour chaque configuration, un modèle LDA était entraîné, puis évalué à l'aide du score de cohérence C_V , qui mesure la cohérence sémantique des mots composant chaque thème. Cette démarche nous a permis d'identifier les paramètres donnant les meilleurs résultats en termes de lisibilité et de pertinence des thèmes extraits. Nous avons ainsi pu constater que certaines combinaisons de paramètres, notamment celles favorisant des répartitions asymétriques, permettaient de générer des thèmes plus distincts et mieux adaptés à notre corpus. L'ensemble des configurations testées a été comparé selon leur score de cohérence, ce qui nous a permis de retenir un modèle final à la fois robuste sur le plan quantitatif et pertinent d'un point de vue interprétatif. Après avoir testé 1500 configurations, voici les paramètres retenus,

Paramètres	Valeur obtenue
K	8
α	asymmetric
β	auto
C_V	0.325

Tableau 2. – Paramètres de la LDA optimisée

IV.6 Modèle Bidirectional Encoder Representations from Transformers (BERT)

IV.6.1 Principe théorique et apports révolutionnaires

Le modèle **BERT** (Bidirectional Encoder Representations from Transformers), introduit par Google en 2018, a marqué une avancée majeure dans le domaine du traitement automatique du langage naturel (NLP). Avant BERT, la plupart des modèles de langage traitaient le texte de manière **unidirectionnelle**, c'est-à-dire en lisant la séquence de mots soit de gauche à droite (comme les RNN classiques ou les premières versions de GPT), soit de droite à gauche. Cette limitation empêchait une compréhension complète et contextuelle des mots, car le sens d'un mot dépend souvent des mots qui le précèdent **et** de ceux qui le suivent.

L'innovation majeure de BERT réside dans sa bidirectionnalité. Cela signifie que pour chaque mot dans une phrase, BERT prend en compte simultanément son contexte à gauche et son contexte à droite. Cette capacité lui permet de construire des représentations vectorielles (embeddings) de mots qui capturent un sens beaucoup plus riche et nuancé, car elles sont informées par l'ensemble de la phrase, et non par une portion limitée.

IV.6.2 Architecture sous-jacente : Le Transformer

BERT repose entièrement sur l'architecture des **Transformers**, introduite par Vaswani et al. (2017) avec leur article « Attention Is All You Need ». Ce qui rend les Transformers si puissants est leur abandon des structures récurrentes (RNN) ou convolutionnelles (CNN) au profit d'un mécanisme unique : le mécanisme d'**attention**.

L'architecture de BERT est constituée d'une pile de plusieurs couches d'encodeurs Transformer. Chaque encodeur (comme illustré dans le schéma ci-dessous) est composé de deux sous-couches principales :

- **Une couche de self-attention multi-tête (Multi-Head Attention):** C'est le cœur du Transformer. Elle permet au modèle de pondérer l'importance de chaque autre mot de la séquence lors de la création de la représentation d'un mot donné. Le terme « multi-tête » signifie que ce mécanisme est appliqué plusieurs fois en parallèle, permettant au modèle de se concentrer sur différentes relations sémantiques ou grammaticales à la fois.
- **Une couche de normalisation et de réseau de neurones à propagation avant (Feed-Forward Network):** Après avoir agrégé les informations via l'attention, cette couche applique une transformation non linéaire à la représentation de chaque mot. Des couches de normalisation sont utilisées pour stabiliser le processus d'entraînement.

L'entrée du modèle BERT n'est pas simplement une suite de mots bruts. Elle est une combinaison de trois types d'embeddings, qui sont sommés pour former la représentation initiale de chaque token :

- **Token Embeddings:** Représentations vectorielles des mots eux-mêmes.
- **Segment Embeddings:** Vecteurs indiquant à quel segment (phrase A ou phrase B) le token appartient. Ceci est crucial pour les tâches impliquant plusieurs phrases, comme la prédiction de la phrase suivante.
- **Positional Embeddings:** Vecteurs qui encodent la position de chaque token dans la séquence. Étant donné que le mécanisme d'attention ne prend pas en compte l'ordre des mots par nature (contrairement aux RNN), ces embeddings sont essentiels pour injecter l'information positionnelle.

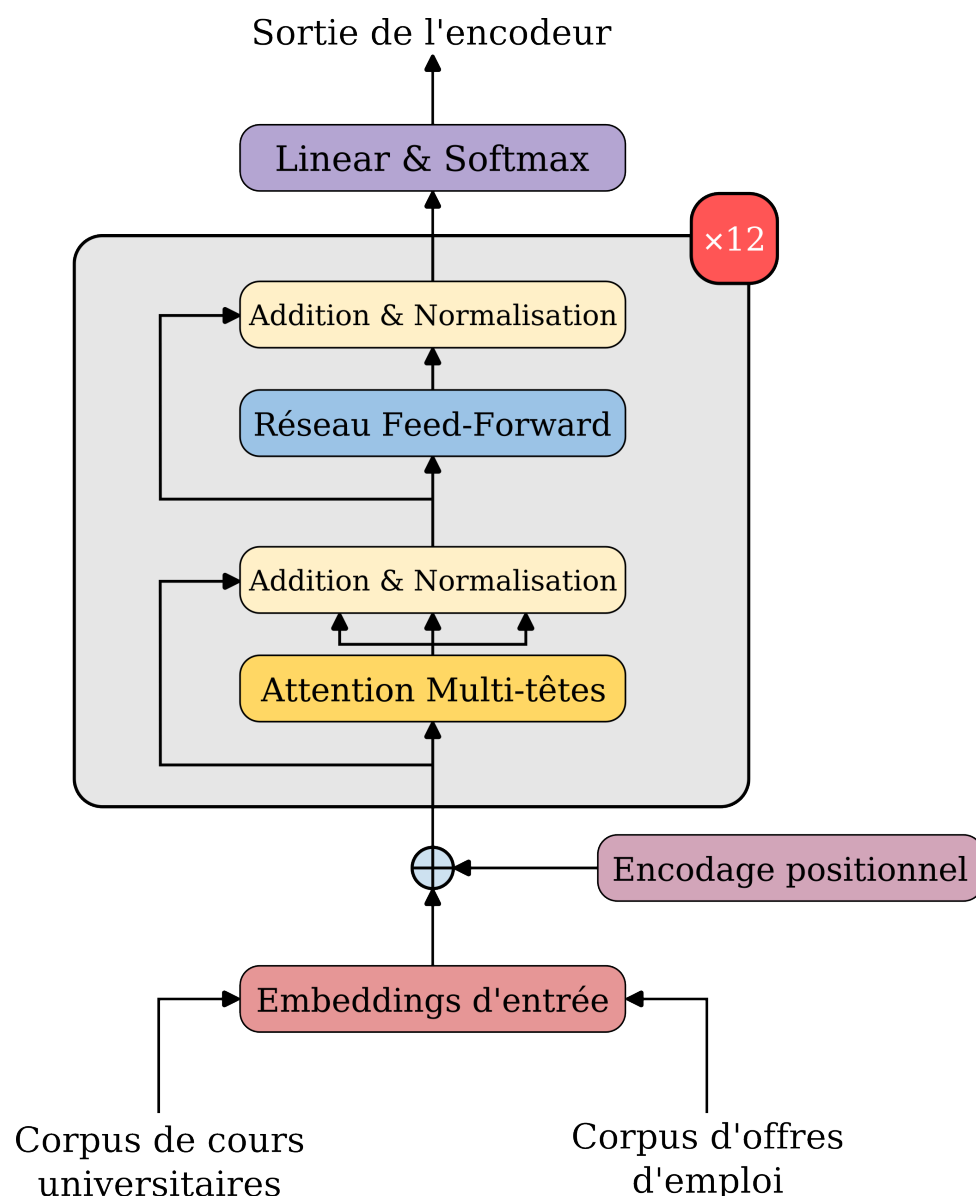


Fig. 2. – Architecture simplifiée d'une couche d'encodeur Transformer, composant de base de BERT

Source: adapté de l'image originale de 'Attention Is All You Need' by Vaswani et al. (2017).

IV.6.3 Objectifs d'apprentissage (Pré-entraînement)

BERT est un modèle qui est « pré-entraîné » sur d'énormes corpus de texte non annotés (comme l'ensemble de Wikipédia et BookCorpus). Ce pré-entraînement se fait à l'aide de deux tâches auto-supervisées, ce qui signifie qu'elles ne nécessitent pas d'annotations humaines explicites :

1. Masked Language Modeling (MLM)

Contrairement aux modèles de langage classiques qui prédisent le mot suivant (unidirectionnel), BERT masque aléatoirement un pourcentage (généralement 15%) des mots dans la phrase. Le modèle apprend ensuite à prédire ces mots masqués en utilisant **l'ensemble de leur contexte bidirectionnel** (les mots avant et après le mot masqué). Ceci force le modèle à développer une compréhension profonde du sens contextuel de chaque mot.

Formellement, étant donné une séquence $x = (x_1, x_2, \dots, x_n)$, on choisit un sous-ensemble $M_{\{1, \dots, n\}}$ de positions à masquer. Le modèle apprend alors à maximiser la vraisemblance des mots masqués :

$$\mathcal{L}_{\text{MLM}} = - \sum_{i \in M} \log P(x_i | x_{\setminus i})$$

où $x_{\setminus i}$ représente la séquence où le token x_i est remplacé par un token spécial [MASK]. Le modèle ne voit que les *tokens* non masqués et doit inférer les masqués.

2. Next Sentence Prediction (NSP)

Cette tâche vise à capturer la relation entre deux phrases. Le modèle reçoit deux segments de texte (A et B) et doit prédire si le segment B est la phrase suivante de A dans le texte d'origine (**IsNext**) ou s'il s'agit d'une phrase aléatoire (**NotNext**). Cette tâche est cruciale pour des applications de compréhension de texte plus longues, comme la question-réponse ou l'inférence textuelle.

Cette tâche est formalisée par :

$$\mathcal{L}_{\text{NSP}} = - \log P(y | A, B)$$

où $y \in \{\text{IsNext}, \text{NotNext}\}$ est la variable binaire à prédire.

La combinaison de ces deux tâches de pré-entraînement permet à BERT d'apprendre des représentations linguistiques riches qui englobent à la fois le sens des mots et les relations entre les phrases.

IV.6.4 Représentation de l'entrée et *tokens* spéciaux

L'entrée d'un modèle BERT suit une structure spécifique pour permettre l'exécution des tâches de pré-entraînement et de **fine-tuning** :

- **[CLS]**: Un token spécial (**classifier token**) placé au début de chaque séquence. Sa représentation vectorielle finale, issue de la dernière couche de BERT, est souvent utilisée comme représentation globale de la séquence pour des tâches de classification de texte.
- **[SEP]**: Un token séparateur utilisé pour délimiter la fin d'une phrase et le début d'une autre, particulièrement pour la tâche NSP où deux phrases (A et B) sont traitées simultanément.
- **[MASK]**: Un token utilisé uniquement pendant le pré-entraînement pour la tâche MLM, remplaçant les mots à prédire.

Exemple de format d'entrée pour BERT :

[CLS] La météo est clémente [SEP] Le soleil brille au jardin [SEP]

Dans cet exemple, le modèle apprendra non seulement le sens des mots, mais aussi la relation entre « La météo est clémente » et « Le soleil brille au jardin ».

IV.6.5 Le mécanisme d'attention bidirectionnelle

Le cœur de BERT réside dans son mécanisme d'attention **bidirectionnelle**, implémenté via la **self-attention multi-tête**. Pour chaque token dans la séquence, le mécanisme de self-attention lui permet de « regarder » tous les autres *tokens* de la même séquence et de calculer une pondération pour chacun d'eux, déterminant ainsi leur pertinence pour la compréhension du token actuel.

Mathématiquement, le calcul de l'attention est souvent décrit par la formule :

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

où :

- **Q (Query)**: Matrice de requêtes, dérivée de la représentation du token actuel.
- **K (Key)**: Matrice de clés, dérivée des représentations de tous les *tokens* de la séquence.
- **V (Value)**: Matrice de valeurs, également dérivée des représentations de tous les *tokens*.
- **d_k** : La dimension des clés, utilisée pour la mise à l'échelle afin d'éviter que les gradients ne deviennent trop grands.

Cette formule décrit comment les requêtes interagissent avec les clés pour produire des scores de similarité, qui sont ensuite normalisés par **softmax** pour obtenir des poids. Ces poids sont appliqués aux valeurs pour produire une représentation contextuelle pour chaque token. La notion de « multi-tête » signifie que ce calcul est effectué plusieurs fois en parallèle avec des projections linéaires différentes pour Q, K, V , permettant de capturer un éventail plus large de relations sémantiques et syntaxiques.

IV.6.6 Avantages et Applications de BERT

La puissance de BERT réside dans sa capacité à être **fine-tuné** (ajusté) pour une multitude de tâches spécifiques en NLP avec un ensemble de données beaucoup plus petit que ce qui serait nécessaire pour entraîner un modèle à partir de zéro. Ses avantages clés incluent :

- **Compréhension contextuelle fine**: Sa nature bidirectionnelle lui permet de saisir les nuances sémantiques des mots en fonction de leur environnement complet.
- **Polyvalence**: Il est extrêmement efficace pour une vaste gamme de tâches NLP, telles que la classification de texte (détection de sentiments, catégorisation), la réponse aux questions (Question Answering - QA), la reconnaissance d'entités nommées (Named Entity Recognition - NER), la détection d'imprécisions grammaticales, et bien d'autres.
- **Transfer Learning efficace**: Le pré-entraînement sur de vastes corpus permet à BERT de servir de base solide, nécessitant peu de données supplémentaires pour s'adapter à une nouvelle tâche (fine-tuning).

IV.6.7 Limitations de BERT

Malgré ses avantages considérables, BERT présente quelques limitations :

- **Coût de pré-entraînement**: Le pré-entraînement de BERT est extrêmement coûteux en termes de temps et de ressources informatiques (nécessitant des accélérateurs comme les TPUs/GPUs). Heureusement, une fois pré-entraîné, il peut être réutilisé.
- **Incapacité à générer du texte**: Contrairement à d'autres architectures basées sur les Transformers (comme GPT, qui sont des modèles autoregressifs), BERT n'est pas conçu pour générer du texte de manière cohérente à partir de zéro. Il est optimisé pour la compréhension.

- **Longueur maximale des séquences:** Les modèles BERT ont une limite sur la longueur des séquences qu'ils peuvent traiter (souvent 512 *tokens*), ce qui peut être un défi pour des documents très longs. Des variantes ont été développées pour gérer ce problème.

IV.6.8 Conclusion et pertinence pour notre étude

BERT a véritablement révolutionné le domaine du NLP en surpassant les approches précédentes sur de nombreux benchmarks. Sa capacité à produire des représentations contextuelles profondes des mots en fait un outil inestimable pour l'analyse sémantique.

Dans le cadre de notre projet sur les disparités entre l'enseignement universitaire et les compétences requises en IA, BERT peut être utilisé de plusieurs manières clés :

- **Évaluation de la similarité sémantique:** Les embeddings de phrases générés par BERT peuvent être utilisés pour calculer la similarité cosinus entre les descriptions d'offres d'emploi et les descriptions de cours. Cela permettra d'identifier précisément les écarts sémantiques au-delà des simples correspondances de mots-clés.
- **Extraction d'informations:** Il pourrait être fine-tuné pour extraire des entités spécifiques (technologies, compétences, rôles) des textes, enrichissant ainsi notre analyse.

En exploitant les représentations vectorielles contextuelles issues de ses couches profondes, BERT nous offre les moyens d'une analyse sémantique avancée, essentielle pour comprendre les nuances des exigences du marché de l'emploi et des offres de formation.

V Méthodes de visualisation

V.1 Analyse en composantes principales (ACP)

V.1.1 Principe

La méthode NLP BERT nous donnent des vecteurs $(X_1, \dots, X_n)$ où $n = 38$ pour les universités et $n = 8$ pour les pays. Chaque X_i est un vecteur ligne composé de 384 coordonnées. Il est donc impossible de visualiser les données surtout dans un espace de taille 384.

Pour ce faire, l'Analyse en Composante Principale (ACP) est une méthode de réduction de dimension afin de visualiser les données sur un plan (2D) ou dans l'espace (3D). Nous détaillons ci-après le principe de l'ACP.

Reprenons notre jeu de données $(X_1, \dots, X_n)$, nous notons pour tout $i \in [1, n]$, $X_i = (X_i^1, X_i^2, \dots, X_i^{384})$, les coordonnées de X_i . Nous notons alors X , la matrice suivante,

$$X = \begin{pmatrix} X_1^1 & \dots & X_1^{384} \\ X_2^1 & \dots & X_2^{384} \\ \vdots & & \vdots \\ X_n^1 & \dots & X_n^{384} \end{pmatrix}.$$

On définit alors la matrice S de covariance empirique par,

$$S = \frac{1}{n} X^\top X,$$

où $X^\top$ désigne la matrice transposée de X . On remarque que S est une matrice symétrique définie positive, elle est donc diagonalisable dans une base orthonormale,

$$S = P \begin{pmatrix} \lambda_1 & & \\ & \ddots & \\ & & \lambda_{384} \end{pmatrix} P^\top,$$

où les λ_i sont les valeurs propres de S , rangées dans l'ordre décroissant ($\lambda_1 \geq \dots \geq \lambda_{\{384\}}$).

Un théorème nous affirme que les j premiers vecteurs propres correspondent à la base dans laquelle la projection des données maximise la variance empirique. Cette projection est donc celle qui contient le plus d'informations sur notre jeu de données. En notant, $P = (p_1, \dots, p_{384})$, la matrice des vecteurs propres, il suffit de réaliser la multiplication suivante,

$$X_{ACP} = X \times (p_1, \dots, p_j).$$

Pour visualiser les données, on choisit $j = 2$ ou $j = 3$, selon que l'on souhaite une représentation en deux ou trois dimensions. Cependant, un point important reste à considérer avant de choisir la valeur de j .

Pour cela, on peut vérifier si la règle du coude est satisfaite. En effet, plus on ajoute de dimensions, plus la variance expliquée augmente. La règle du coude consiste à tracer la part de variance expliquée en fonction du nombre de dimensions, et à choisir j au point où cette courbe présente un « coude », indiquant un gain marginal réduit au-delà.

V.1.2 Implémentation et résultats

V.1.2.1 BERT

L'implémentation sous Python de l'ACP est fait en utilisant la bibliothèque `sklearn.decomposition` qui offre un objet `PCA`⁶ (pour *Principal Component Analysis*) qui réalise la réduction pour nous.

V.1.2.1.1 Choix de la dimension

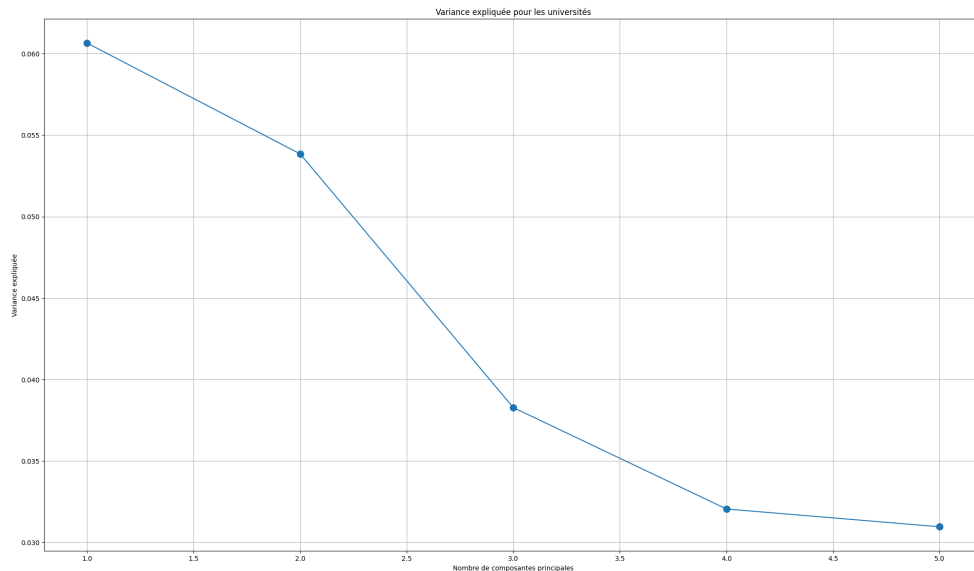


Fig. 3. – Variance expliquée pour les universités

⁶<https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.PCA.html>

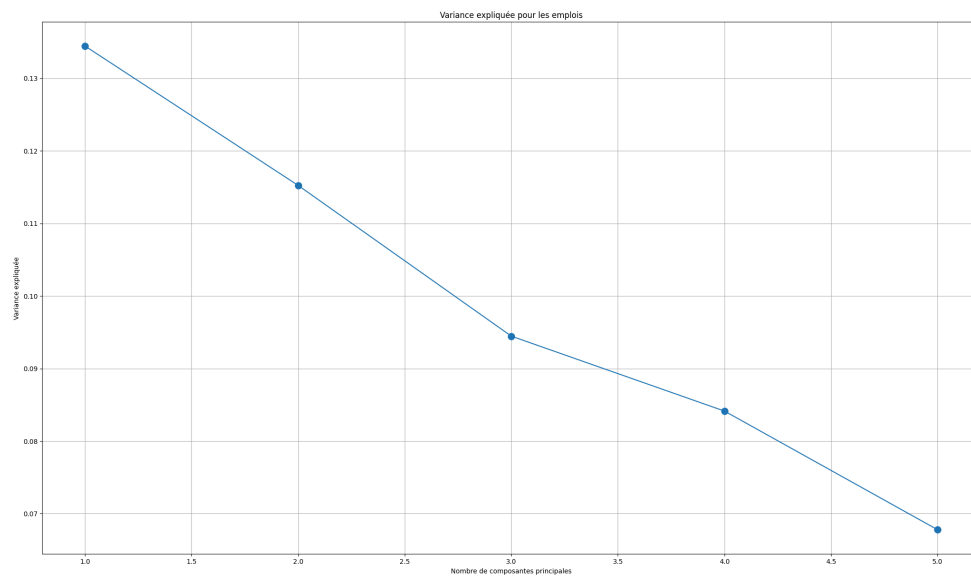


Fig. 4. – Variance expliquée pour les emplois

On remarque que sur la Fig. 3, la règle du coude nous donne que $j = 3$. De même, la règle du coude sur la Fig. 4 nous donne $j = 3$, malgré le coude un peu moins prononcé.

V.1.2.1.2 Visualisation

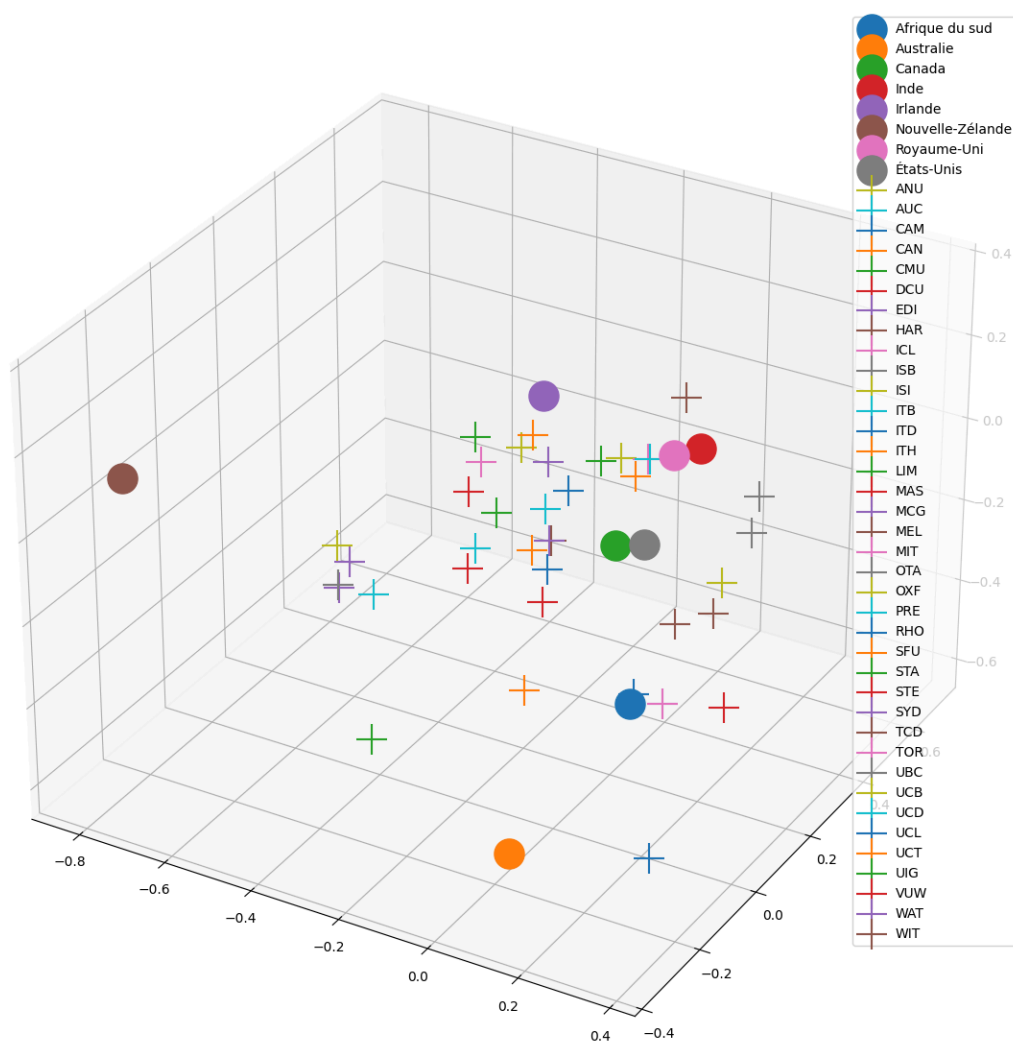


Fig. 5. – Visualisation des données en trois dimensions

Nous remarquons que deux pays sont très éloignés du reste du nuage de points : la Nouvelle-Zélande et l'Australie. En effet, comme vu précédemment le *scraping* de la Nouvelle-Zélande n'ayant résulté que de 223 d'offres d'emplois au lieu des 1000 attendus, il est normal que ce point se trouve à l'écart du reste.

Grâce à l'ACP, nous espérons trouver des résultats similaires lors du calcul de la similarité.

La bibliothèque `sklearn.decomposition` offre une méthode facile à implémenter et donc il est également facilement possible de vérifier en deux dimensions s'il n'y a pas de points singuliers.

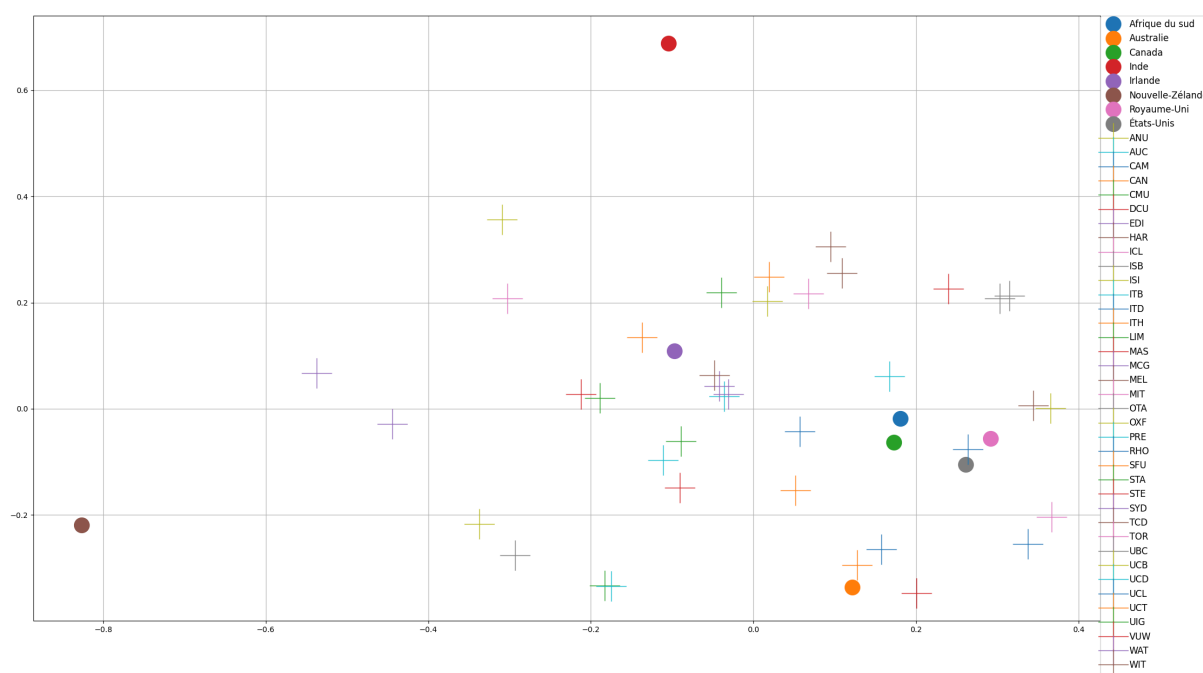


Fig. 6. – Visualisation des données en deux dimensions

On remarque comme tout à l'heure que la Nouvelle-Zélande est un point à part. De plus, l'Inde est devenu un point singulier. On remarque tout de suite que cela est dû à la part de variance expliquée qui ne « réalise un coude » qu'à partir de la troisième dimension ajoutée.

V.1.2.2 LDA

V.1.2.2.1 Visualisation avec pyLDavis

Pour interpréter les résultats du modèle LDA, nous avons utilisé la bibliothèque Python `pyLDavis`⁷ qui permet de visualiser de manière interactive les thèmes identifiés. Cette visualisation offre une projection bidimensionnelle des topics sur un plan, construite à partir d'une réduction de dimension (par multidimensional scaling, MDS), où chaque cercle représente un thème.

La taille des cercles indique la proportion du corpus représentée par le thème, tandis que leur position reflète les relations sémantiques entre les thèmes : plus deux thèmes sont proches, plus ils partagent de similarités lexicales. À droite de la visualisation, une liste des termes les plus représentatifs est affichée pour chaque thème, avec un ajustement possible (via un curseur) entre fréquence globale et exclusivité pour affiner l'interprétation.

L'utilisation de `pyLDavis` nous a permis :

- d'évaluer la cohérence thématique des résultats,
- de détecter d'éventuels recouvrements ou redondances entre les thèmes,
- et de guider le choix du nombre optimal de topics K en identifiant les clusters bien séparés et les thèmes trop proches.

Cette étape de visualisation est cruciale car elle complète l'approche quantitative (par exemple les scores de cohérence) par une inspection qualitative des thèmes, assurant que les résultats sont non seulement statistiquement valides mais aussi interprétables et exploitables pour l'analyse comparative entre formations et marché de l'emploi.

⁷<https://pyldavis.readthedocs.io/en/latest/>

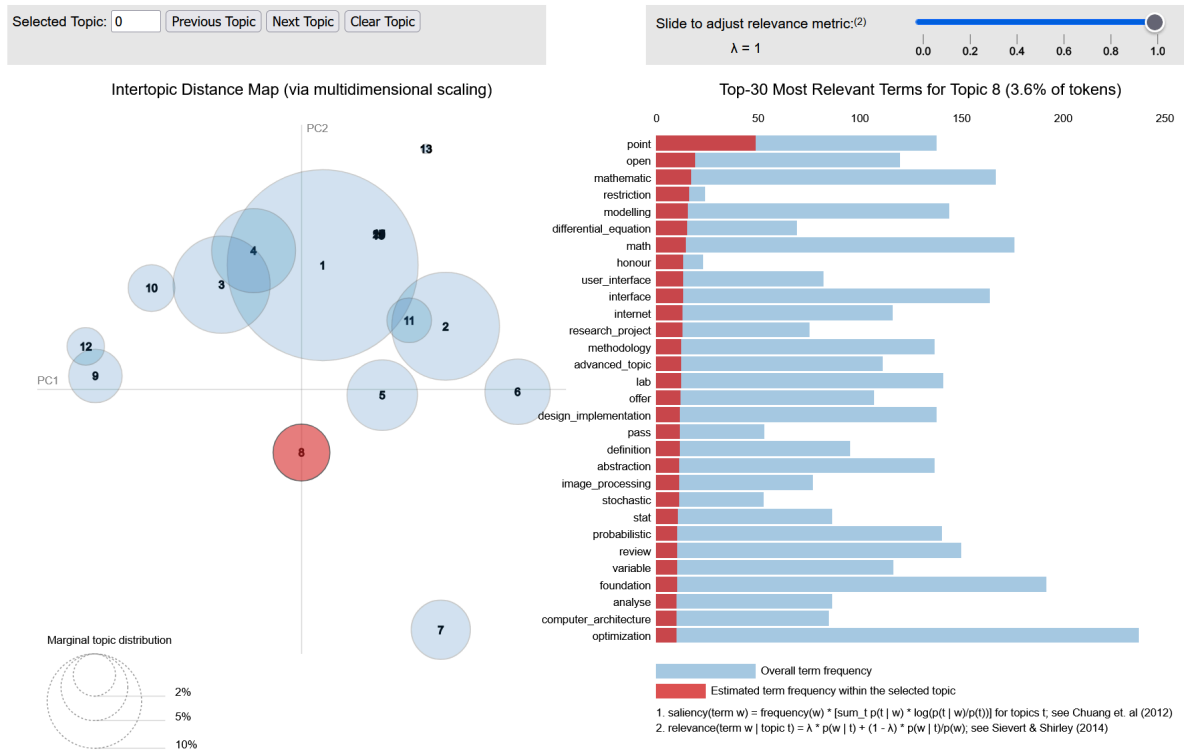


Fig. 7. – Exemple de visualisation avec pyLDAvis sur les cours d’universités

Nous voyons sur la Fig. 7 que les cercles sont bien séparés (les topics sont bien différenciés), distincts et de faible rayons (les topics sont spécialisés). De plus, on remarque que le topic 8 contient des termes comme « mathématiques », « équations différentielles », « stochastique », ..., des termes très importants dans le domaine du *Machine Learning* et donc de l’IA.

V.2 Uniform Manifold Approximation and Projection (UMAP)

V.2.1 Principe

Contrairement à l’ACP qui est une méthode linéaire, **UMAP** est une technique de réduction de dimension **non-linéaire**. Son principal avantage est de préserver à la fois la structure locale et globale des données. UMAP suppose que les données sont échantillonnées à partir d’un « manifold » (une variété topologique) de faible dimension et vise à construire une représentation de ce manifold dans un espace de dimension inférieure.

La méthode procède en deux étapes principales :

1. **Construction d’un graphe flou en haute dimension** : UMAP construit un **graphe pondéré** sur les données de haute dimension. Pour chaque point x_i , on détermine ses k plus proches voisins (où k est défini par l’hyperparamètre `n_neighbors`). La similarité entre deux points x_i et x_j est ensuite transformée en une probabilité de connectivité floue, p_{ij} , en utilisant une fonction d’activation non-linéaire (souvent basée sur une exponentielle de la distance). Cette probabilité est calculée en considérant la distance entre les points et un rayon local adaptatif pour chaque point, ce qui permet de donner plus d’importance aux voisinages proches. L’objectif est de s’assurer que les distances dans le graphe reflètent la structure des données sur le manifold. Formellement, la probabilité de connectivité p_{ij} de x_j dans le voisinage de x_i est donnée par:

$$p_{i|j} = \exp\left(-\frac{d(x_i, x_j) - \rho_i}{\sigma_i}\right)$$

où $d(x_i, x_j)$ est la distance entre x_i et x_j , ρ_i est la distance au k -ième plus proche voisin de x_i (ajustée pour garantir une connectivité minimale), et σ_i est une normalisation locale.

Les probabilités sont ensuite symétrisées pour obtenir $p_{ij} = p_{i|j} + p_{j|i} - p_{i|j} \cdot p_{j|i}$, assurant ainsi la création d'un graphe non orienté.

2. **Optimisation de la représentation en basse dimension** : UMAP cherche ensuite à construire un graphe similaire en basse dimension (par exemple, 2D ou 3D). Pour chaque paire de points y_i et y_j dans l'espace de basse dimension, une probabilité de connectivité q_{ij} est calculée de manière similaire, mais en utilisant une distribution t-Student (ou une autre distribution à queue lourde) pour mieux représenter les distances :

$$q_{ij} = \frac{1}{1 + a(d(y_i, y_j))^{2b}}$$

où a et b sont des paramètres dérivés de `min_dist`. L'algorithme minimise la **divergence de Kullback-Leibler** (ou une fonction de perte basée sur l'entropie croisée) entre les probabilités de haute dimension (p_{ij}) et celles de basse dimension (q_{ij}), afin de trouver la meilleure disposition des points en basse dimension qui préserve la structure de similarité. La fonction de perte totale que UMAP minimise est :

$$\text{Loss} = \sum_{i \neq j} \left[p_{ij} \log\left(\frac{p_{ij}}{q_{ij}}\right) + (1 - p_{ij}) \log\left(\frac{1 - p_{ij}}{1 - q_{ij}}\right) \right]$$

Cette minimisation est réalisée à l'aide d'une descente de gradient stochastique.

Les hyperparamètres clés de UMAP incluent :

- `n_neighbors` : Ce paramètre contrôle le nombre de voisins à considérer pour chaque point lors de la construction du graphe. Une petite valeur met l'accent sur la préservation de la structure locale (clusters compacts), tandis qu'une grande valeur permet de mieux capturer la structure globale (relations entre les clusters).
- `min_dist` : Il définit la distance minimale autorisée entre les points dans l'espace de faible dimension. Une valeur faible rend les clusters plus denses et les points plus groupés, tandis qu'une valeur élevée permet une plus grande dispersion des points au sein des clusters.
- `metric` : La mesure de distance utilisée pour calculer les similarités en haute dimension (par défaut, la distance euclidienne).

V.2.2 Implémentation et résultats

Nous avons utilisé la bibliothèque `umap-learn`⁸ pour l'implémentation de UMAP. Les visualisations obtenues via UMAP nous permettent d'observer la distribution des embeddings pour les offres d'emploi et les cours universitaires.

⁸<https://umap-learn.readthedocs.io/en/latest/>

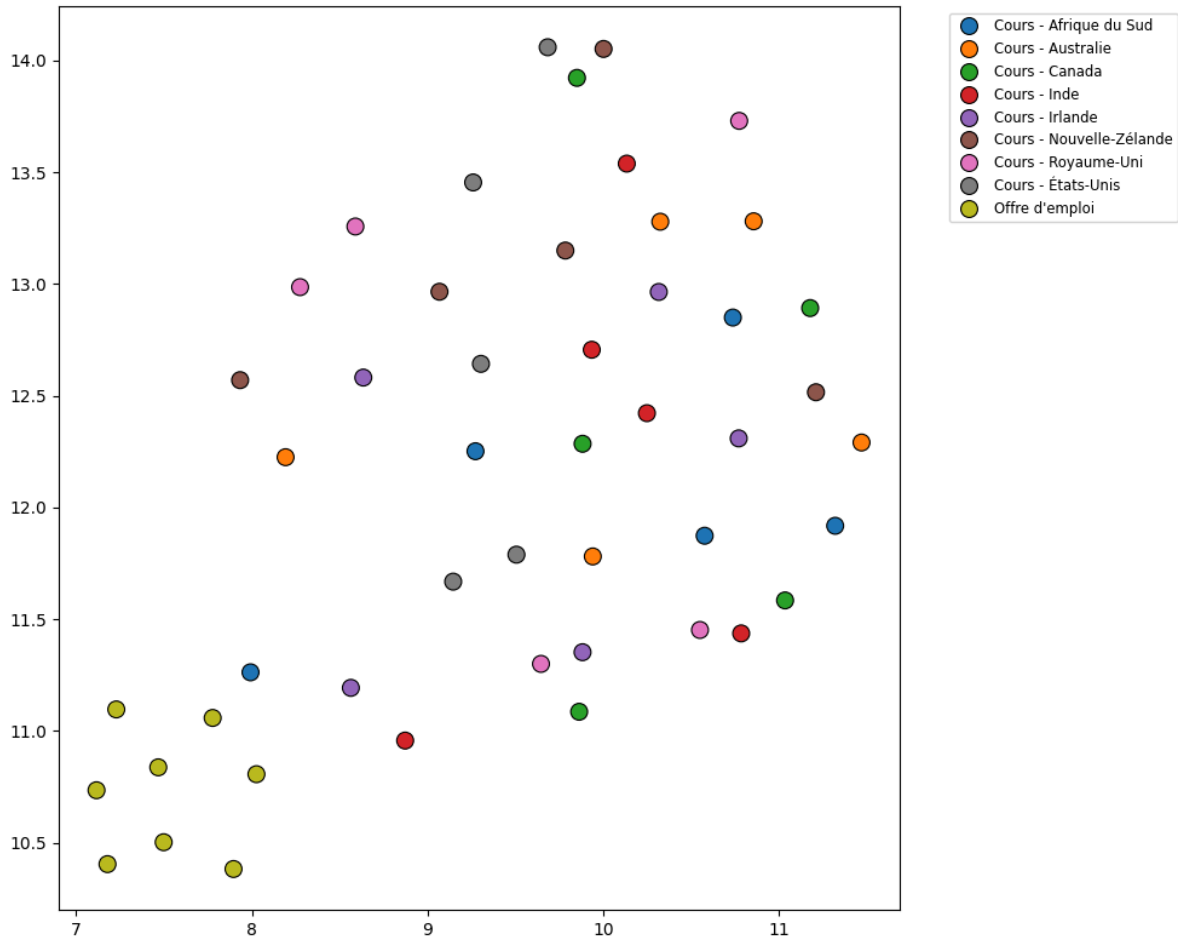


Fig. 8. – Visualisation UMAP des embeddings BERT

Sur la Fig. 8, qui représente la projection UMAP combinée des universités (distinguées par pays) et des offres d'emploi, nous pouvons faire les observations suivantes :

- **Séparation globale** : On observe une distinction nette entre le groupe des « Offres d'emploi » (points jaunes) et le groupe des « Cours » (points de différentes couleurs pour les pays). Les offres d'emploi sont regroupées dans la partie inférieure gauche du graphique, tandis que les cours universitaires occupent la partie supérieure. Cela suggère une différence sémantique fondamentale et bien préservée entre le contenu des offres d'emploi et celui des descriptions de cours.
- **Structure interne des cours** : Au sein du groupe des cours universitaires, les points de différentes couleurs (représentant les pays) ne forment pas des clusters parfaitement isolés. Ils sont plutôt entremêlés, bien que certaines tendances puissent être aperçues. Par exemple, les « Cours - Afrique du Sud » (bleu foncé) et « Cours - Canada » (orange) semblent avoir une certaine dispersion, tandis que d'autres pays comme l'« Inde » (rouge) ou le « Royaume-Uni » (violet) montrent des regroupements plus diffus. Cela indique que, malgré les différences de contenu liées aux spécificités nationales, il existe un chevauchement thématique significatif entre les cours de différentes universités, indépendamment de leur pays d'origine.
- **Homogénéité des offres d'emploi** : Le cluster des « Offres d'emploi » est relativement compact et homogène, ce qui suggère une cohérence thématique parmi les descriptions de poste collectées.
- **Interprétation des distances** : La distance relative entre le groupe des offres d'emploi et celui des cours, ainsi que la dispersion au sein des clusters de cours, reflètent la similarité

sémantique de leurs contenus. UMAP, en préservant la structure globale, indique que les domaines des cours et des emplois sont distincts mais pas complètement disjoints, avec une certaine proximité sémantique générale qui justifie leur présence sur le même graphique.

V.3 t-Distributed Stochastic Neighbor Embedding (t-SNE)

V.3.1 Principe

t-SNE est une autre méthode de réduction de dimension **non-linéaire**, particulièrement reconnue pour sa capacité à visualiser des données de haute dimension en 2 ou 3 dimensions en mettant en évidence la structure des **clusters locaux**.

Le principe de t-SNE est de convertir les distances entre les points en probabilités de similarité. Il crée une distribution de probabilité des paires de points en haute dimension et une distribution similaire en basse dimension. L'algorithme minimise ensuite la **divergence de Kullback-Leibler** entre ces deux distributions, ce qui le pousse à rapprocher les points qui étaient proches en haute dimension et à éloigner ceux qui étaient distants.

Formellement, pour deux points x_i et x_j en haute dimension, la similarité est modélisée par une distribution gaussienne centrée sur x_i . La probabilité conditionnelle $p_{i|j}$ que x_j soit un voisin de x_i est donnée par :

$$p_{j|i} = \frac{\exp\left(\frac{-\|x_i - x_j\|^2}{2\sigma_i^2}\right)}{\sum_{k \neq i} \exp\left(\frac{-\|x_i - x_k\|^2}{2\sigma_i^2}\right)}$$

où σ_i est une variance spécifique à chaque point x_i , déterminée de manière à ce que l'entropie de P_i (la distribution de $p_{j|i}$ pour un x_i fixe) corresponde à une **perplexity** donnée. La similarité conjointe p_{ij} est ensuite symétrisée : $p_{ij} = \frac{p_{j|i} + p_{i|j}}{2N}$, où N est le nombre de points.

Dans l'espace de basse dimension, pour les points correspondants y_i et y_j , t-SNE utilise une distribution t-Student à un degré de liberté (aussi appelée distribution de Cauchy) pour modéliser la similarité $q_{i,j}$. Cette distribution est choisie car elle a des « queues lourdes », ce qui aide à atténuer l'effet du *crowding problem* (problème d'encombrement) où tous les points se tassent au centre en basse dimension :

$$q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq l} (1 + \|y_k - y_l\|^2)^{-1}}$$

L'algorithme de t-SNE minimise la divergence de Kullback-Leibler entre la distribution P (haute dimension) et la distribution Q (basse dimension) :

$$\text{KL}(P\|Q) = \sum_{i \neq j} p_{ij} \log\left(\frac{p_{ij}}{q_{ij}}\right)$$

La minimisation de cette fonction est réalisée par une descente du gradient.

t-SNE excelle à préserver les voisinages locaux des points, ce qui signifie que les points qui sont proches en haute dimension resteront généralement proches en basse dimension. Cependant, il est important de noter que t-SNE est moins fiable pour préserver les distances globales ; la taille des clusters ou la distance entre eux en basse dimension ne reflète pas toujours fidèlement leur taille ou leur distance réelle en haute dimension.

L'hyperparamètre clé de t-SNE est la **perplexity**. Elle peut être interprétée comme une mesure du nombre de voisins « efficaces » autour de chaque point. Une valeur typique se situe entre 5 et 50. Des valeurs trop basses peuvent entraîner une focalisation excessive sur le bruit, tandis que des valeurs trop élevées peuvent masquer des structures fines en se concentrant trop sur les voisinages lointains.

V.3.2 Implémentation et résultats

L'implémentation de t-SNE se fait via la bibliothèque `sklearn.manifold.TSNE`⁹. La projection t-SNE combinée des cours et des offres d'emploi est présentée sur la Fig. 9.

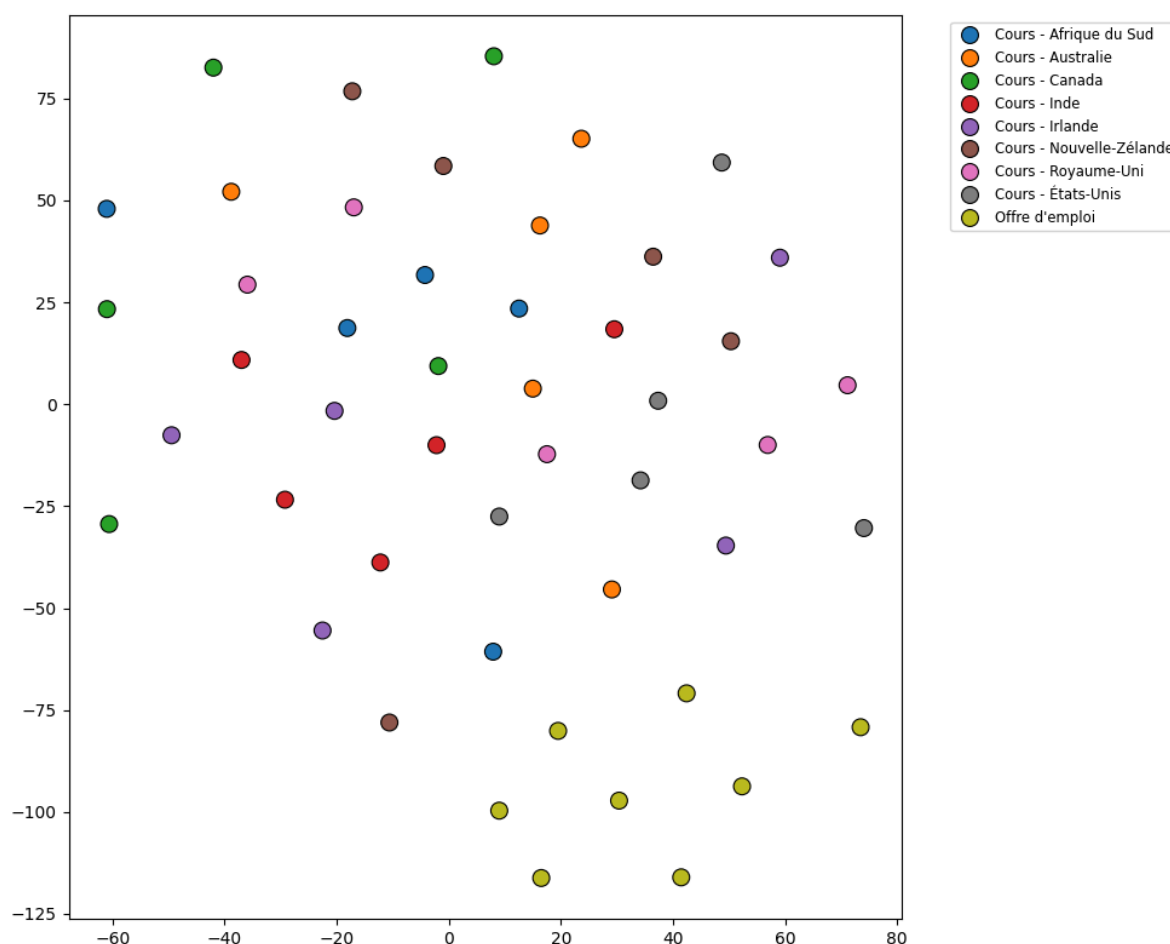


Fig. 9. – Visualisation t-SNE des embeddings BERT

En comparant avec les résultats d'UMAP, t-SNE révèle une organisation des données différente, typique de son approche, même si certaines conclusions restent identiques :

- **Clusters locaux plus distincts** : Sur la figure, t-SNE semble mieux séparer les différents groupes de « Cours » par pays. On observe des agrégations plus distinctes pour chaque catégorie de cours, ce qui suggère que t-SNE a davantage mis l'accent sur la structure locale des données. Par exemple, les cours de l'Afrique du Sud et de la Nouvelle-Zélande, qui étaient un peu plus mélangés dans UMAP, apparaissent ici comme des groupes plus compacts.
- **Maintien de la séparation cours / offres d'emploi** : Comme avec UMAP, t-SNE maintient une séparation claire entre le groupe des « Offres d'emploi » (jaune) et celui des

⁹<https://scikit-learn.org/stable/modules/generated/sklearn.manifold.TSNE.html>

« Cours ». Les offres d’emploi se regroupent toujours dans la partie inférieure gauche du graphique, mais de manière plus resserrée.

- **Distances inter-clusters moins fiables** : Bien que les clusters individuels soient plus nets, la distance entre eux sur le graphique t-SNE n’est pas directement interprétable comme une distance sémantique en haute dimension. t-SNE optimise la préservation des voisinages et non les distances globales, ce qui peut entraîner une compression ou une étirement artificiel des espaces entre les groupes.
- **Structure globale différente** : La disposition générale des clusters de pays et du cluster d’emplois est moins structurée globalement que sur le graphique UMAP. UMAP, visant à préserver la structure globale, offre une vue plus cohérente de la relation globale entre tous les points. t-SNE, en revanche, excelle à montrer les « boules » de similarité locale.

V.4 Synthèse des méthodes de réduction de dimension

En résumé, les trois méthodes d’analyse de dimension que nous avons explorées présentent des caractéristiques distinctes et des applications différentes :

- L’**ACP** est une méthode linéaire, rapide et efficace pour la préservation de la variance globale, idéale pour identifier les directions principales de variabilité des données.
- **UMAP** est une méthode non-linéaire qui se distingue par sa capacité à préserver à la fois la structure locale et globale, souvent privilégiée pour les jeux de données volumineux grâce à sa vitesse d’exécution.
- Le **t-SNE** est également non-linéaire, excellent pour la visualisation de clusters locaux et la révélation de structures complexes, bien qu’il puisse être plus lent sur de grands ensembles de données et moins fiable pour les interprétations de distances inter-clusters.

VI Similarité sémantique

VI.1 Similarité cosinus

Nous avons commencé par étudier la similarité cosinus. Il s’agit de la méthode la plus fréquemment utilisée pour mesurer la similarité de deux documents. Soient X_1 et X_2 deux vecteurs représentant les corpus, la similarité cosinus se définit par :

$$\text{Similarité}_{\text{cosinus}}(X_1, X_2) = \cos \theta = \frac{\langle X_1, X_2 \rangle}{\|X_1\| \|X_2\|},$$

où le produit scalaire $\langle X_1, X_2 \rangle = X_1^\top X_2 = \sum_{i=1}^n X_1^{(i)} X_2^{(i)}$, et la norme est la norme euclidienne. θ représente l’angle entre les deux vecteurs X_1 et X_2 dans l’espace vectoriel.

Par construction, cette mesure varie entre -1 et 1 , où 1 indique une parfaite similarité directionnelle, 0 une orthogonalité (absence de similarité), et -1 une opposition parfaite :

- Si $\theta = 0^\circ$, les vecteurs pointent exactement dans la même direction $\rightarrow \cos(0) = 1$, **similarité maximale**,
- Si $\theta = 90^\circ$, les vecteurs sont orthogonaux $\rightarrow \cos(90^\circ) = 0$, **pas de similarité**,
- Si $\theta = 180^\circ$, les vecteurs pointent dans des directions opposées $\rightarrow \cos(180^\circ) = -1$, **opposition parfaite**

Toutefois, dans le cadre de notre travail avec les représentations issues de modèles comme BERT, les vecteurs produits sont situés dans le même quadrant de l’espace vectoriel : par conséquent, la similarité cosinus mesurée se situe pratiquement toujours dans l’intervalle $[0, 1]$. Ainsi, une valeur proche de 1 reflète une forte proximité sémantique, tandis qu’une valeur proche de 0 traduit une différence marquée.

VI.2 Divergence de Jensen-Shannon

La **divergence de Jensen-Shannon** (JS) est une mesure de similarité particulièrement adaptée pour comparer des distributions de probabilité, comme celles générées par des modèles thématiques (dans notre étude, LDA) ou des représentations probabilistes de texte. Contrairement à la divergence de **Kullback-Leibler** (KL), qui est asymétrique et peut être infinie si les distributions n'ont pas le même support, la divergence de Jensen-Shannon est **symétrique** et toujours finie, ce qui en fait un choix privilégié en traitement automatique des langues (NLP).

Soient P et Q deux distributions de probabilité sur un espace commun, on définit leur moyenne $M = \frac{1}{2}(P + Q)$. La divergence de Jensen-Shannon est donnée par :

$$JS(P\|Q) = \frac{1}{2}KL(P\|M) + \frac{1}{2}KL(Q\|M),$$

où $KL(P\|M)$ désigne la divergence de Kullback-Leibler entre P et M , définie comme :

$$KL(P\|M) = \sum_i P(i) \frac{\log(P(i))}{M(i)}.$$

Cette mesure évalue donc à quel point chaque distribution s'écarte de leur moyenne commune. Comme elle est symétrique ($JS(P\|Q) = JS(Q\|P)$) et bornée dans l'intervalle $[0, \log 2]$ (souvent normalisée entre 0 et 1), elle est interprétable comme une mesure de dissimilarité, où 0 indique deux distributions identiques et 1 (ou $\log 2$) une dissimilarité maximale.

En pratique, pour calculer la **similarité** à partir de la divergence JS, on a utilisé $1 - JS(P\|Q)$, qui renvoie une valeur de proximité : plus proche de 1 si les distributions sont semblables, proche de 0 si elles sont très différentes. Dans le cadre de notre projet, cela permet de comparer, par exemple, les distributions thématiques des *syllabi* universitaires et des offres d'emploi, en capturant non seulement les correspondances exactes, mais aussi les similarités partielles au niveau des thèmes.

VI.3 Indice de Jaccard

L'**indice de Jaccard**, également connu sous le nom de **coefficient de Jaccard**, est une mesure statistique utilisée pour évaluer la similarité et la diversité d'ensembles d'échantillons. Il est particulièrement utile lorsque l'on souhaite comparer la composition d'ensembles de mots, de concepts ou d'éléments discrets.

Soient deux ensembles A et B . L'indice de Jaccard est défini comme le rapport entre la taille de l'intersection des ensembles et la taille de leur union :

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

où :

- $|A \cap B|$ représente le nombre d'éléments communs aux deux ensembles A et B .
- $|A \cup B|$ représente le nombre total d'éléments uniques présents dans l'un ou l'autre des ensembles A ou B .

La valeur de l'indice de Jaccard est toujours comprise entre 0 et 1, inclus :

- $J(A, B) = 0$: Indique qu'il n'y a aucun élément commun entre les deux ensembles. Les ensembles sont totalement dissemblables.

- $J(A, B) = 1$: Indique que les deux ensembles sont identiques. Tous leurs éléments sont communs.
- $0 < J(A, B) < 1$: La valeur reflète le degré de chevauchement. Plus la valeur est proche de 1, plus les ensembles sont similaires.

La **distance de Jaccard**, complémentaire à l'indice de Jaccard, est souvent utilisée pour mesurer la dissimilarité : $D_{J(A,B)} = 1 - J(A, B)$. Elle varie également entre 0 et 1.

Dans le cadre de notre analyse des disparités, l'indice de Jaccard est une méthode directe et intuitive pour :

- **Comparer les ensembles de mots-clés extraits** des offres d'emploi et des programmes universitaires. Par exemple, si nous avons une liste de compétences requises dans un job et une liste de compétences enseignées dans un cours, l'indice de Jaccard peut quantifier leur degré de chevauchement.
- **Évaluer la similarité entre les vocabulaires** associés à des thèmes identifiés par des modèles comme la LDA ou Top2Vec. Si deux thèmes sont représentés par des ensembles de mots clés, Jaccard peut indiquer à quel point ces thèmes sont similaires au niveau lexical.

C'est une mesure simple à comprendre et à interpréter, particulièrement efficace pour des comparaisons d'ensembles non-ordonnés d'éléments discrets.

VI.4 Utilisation sur nos modèles

VI.4.1 BERT

Similarité cosinus **Utilisable**

BERT produit des embeddings (qui sont des vecteurs) pour les mots, les phrases ou les corpus. La similarité cosinus est la mesure standard pour évaluer la proximité directionnelle de ces vecteurs dans l'espace d'embedding. On peut donc directement comparer les embeddings des offres d'emploi avec ceux des *syllabi* de cours.

Divergence de Jensen-Shannon **Non Utilisable**

La divergence de Jensen-Shannon est conçue pour comparer des distributions de probabilité.

Les embeddings de BERT sont des vecteurs de nombres réels qui ne sont pas intrinsèquement des distributions de probabilité (ils n'ont pas une somme égale à 1 et ne représentent pas des masses de probabilité).

On ne peut donc pas utiliser cette méthode avec BERT.

Indice de Jaccard **Non Utilisable**

L'indice de Jaccard est une mesure de similarité entre des ensembles (de données discrètes).

Les embeddings de BERT sont des vecteurs continus, pas des ensembles discrets.

On pourrait utiliser cette méthode si on extrayait des concepts ou des mots-clés à partir des embeddings de BERT. Mais ceci est impossible car les embeddings sont bijectifs : **plusieurs termes qui peuvent sembler ne rien avoir en commun peuvent avoir le même embedding**. Il est alors impossible d'en tirer un topic global.

VI.4.2 LDA

Similarité cosinus **Utilisable**

On peut utiliser cette méthode à la fois pour comparer les distributions de topics et les distributions de mots dans les topics.

- **Comparaison de corpus via les topics**

Chaque document est représenté par un vecteur de probabilité sur les K topics (par exemple $[0.1, 0.6, 0.3]$ pour 3 topics). On peut calculer la similarité cosinus entre ces vecteurs de distribution de topics du corpus d'offres d'emploi et celle des *syllabi* de cours, pour voir s'ils partagent des proportions similaires de sujets.

- **Comparaison des topics entre eux**

Chaque topic est une distribution de probabilités sur tous les mots du vocabulaire. On peut calculer la similarité cosinus entre les vecteurs de ces distributions de mots de deux topics pour voir à quel point ils sont sémantiquement proches.

Dans notre étude, seule la comparaison entre corpus nous intéresse.

Divergence de Jensen-Shannon Utilisable

La LDA produit des distributions de probabilité (distribution de topics par document, distribution de mots par topics). La divergence de Jensen-Shannon est idéale pour comparer ces distributions.

Comme pour la similarité cosinus, elle permet de comparer :

- les distributions de topics de deux corpus
- les distributions de mots de deux topics

Une nouvelle fois, on s'intéressera à la distribution de topics dans deux corpus : offres d'emploi et *syllabi* de cours.

Indice de Jaccard Utilisable Indirectement

On peut utiliser cette mesure, mais indirectement : la LDA elle-même ne produit pas directement des ensembles.

On peut utiliser cet indice si on :

- compare les ensembles des N mots les plus probables de deux topics LDA pour voir leur chevauchement lexical.
- compare des ensembles de documents assignés majoritairement à un topic donné.
- compare des ensembles de mots-clés que l'on a pu extraire ou lister (par exemple, à partir des top mots d'un topic LDA).

Cependant, nous n'avons pas réussi à l'implémenter, nous ne l'étudierons pas au sein de ce projet.

VI.4.3 Récapitulatif

Modèle utilisé	BERT	LDA
Similarité cosinus	Utilisable	Utilisable
Divergence de Jensen-Shannon	Non utilisable	Utilisable
Indice de Jaccard	Non utilisable	Utilisable indirectement (non abordé)

VII Résultats

VII.1 Étude préliminaire : analyse fréquentielle

Dans un premier temps, on peut considérer comme analyse simple le fait de comparer les fréquences de certains mots importants choisis à l'avance et regroupés dans une liste. Pour notre *whitelist*, nous nous sommes concentrés sur des termes techniques courants dans les thèmes de l'intelligence artificielle ou bien la science des données composée de 143 mots.

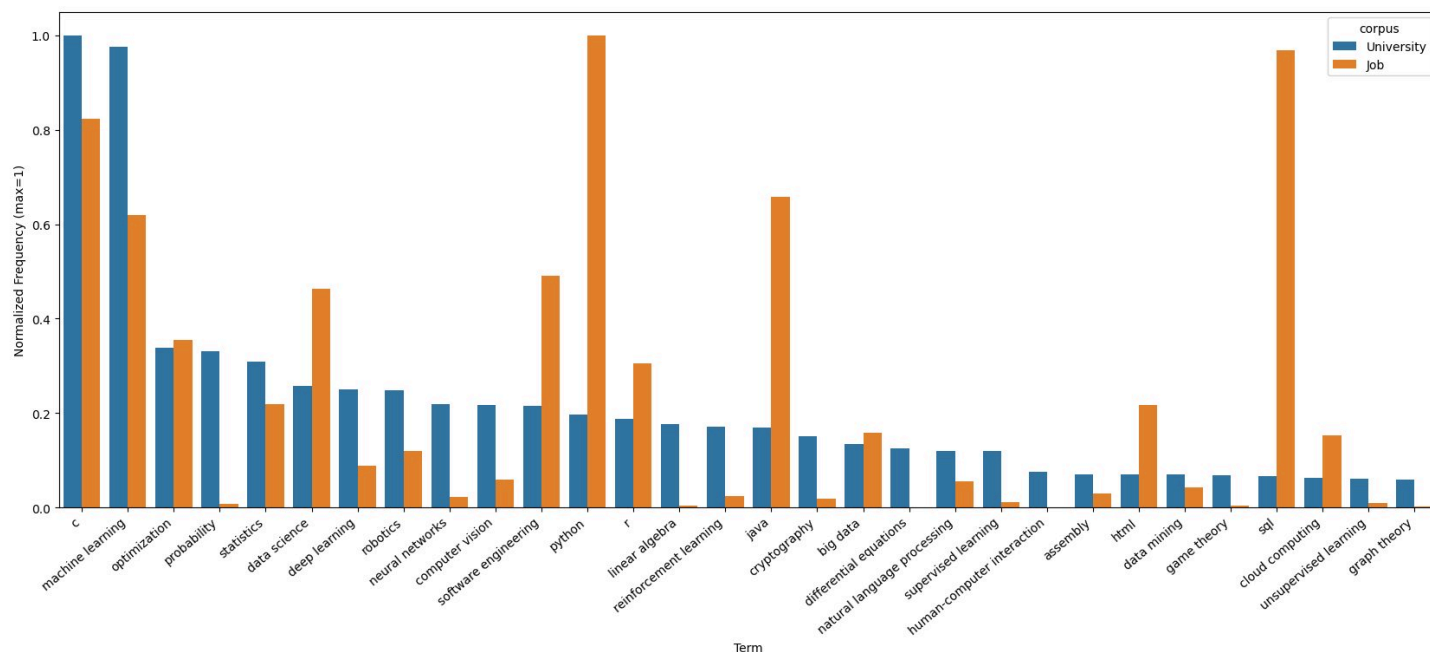


Fig. 10. – Top 30 des mots de la *whitelist* en fréquence dans le corpus Jobs

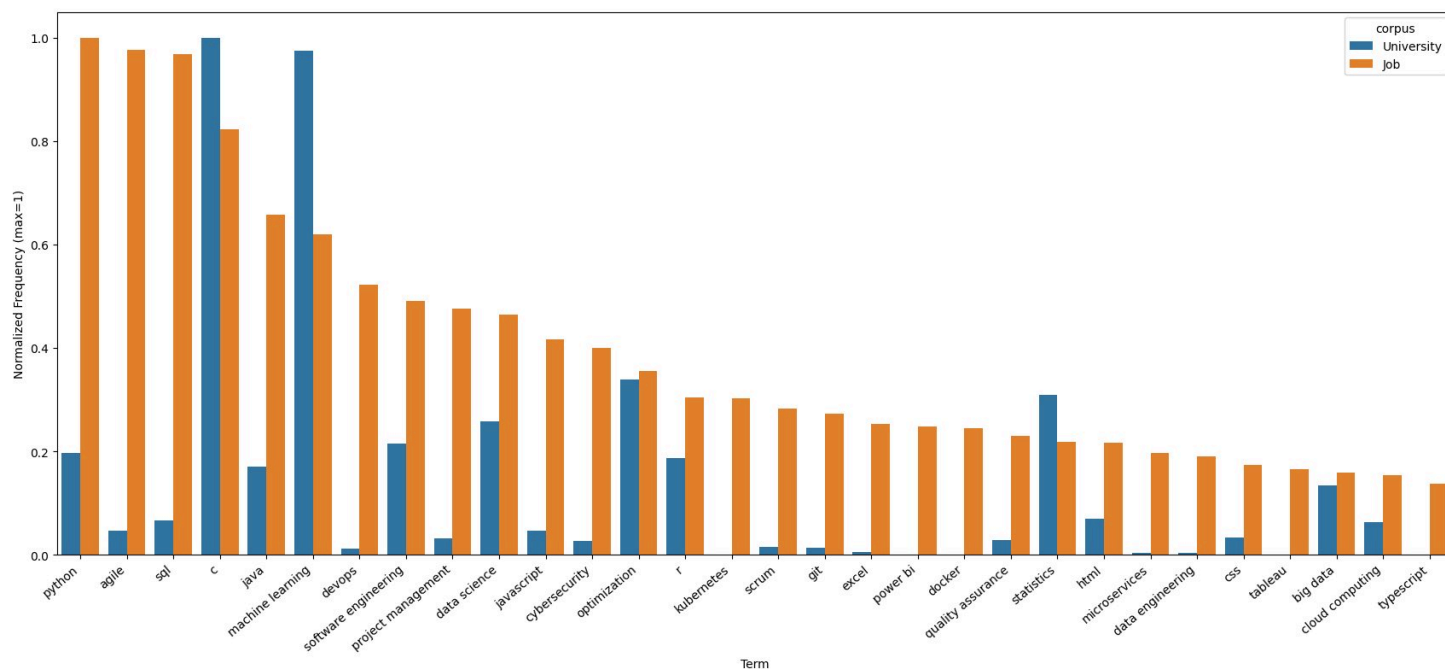


Fig. 11. – Top 30 des mots de la *whitelist* en fréquence dans le corpus Jobs

Cette analyse préliminaire nous indique que sur les domaines techniques, les cours d'université se concentrent sur un ensemble de bases et de fondamentaux, notamment les outils mathéma-

tiques et langages de programmations. Si les entreprises nécessitent elles aussi des compétences spécifiques aux langages de programmation, on remarque ainsi que parmi la liste des mots les plus communs de la liste pour les entreprises, on trouve de nombreux noms de langages de programmation comme c, java ou r. Cependant les entreprises nécessitent souvent des compétences spécifiques à certains outils utilisés, comme git, power bi, docker ou excel qui sont très largement délaissées par les universités. Sur des enjeux globaux comme le *machine learning*, la *data science* ou bien l'optimisation, ces sujets sont à peu près traités en adéquation avec la demande des entreprises. Enfin, de nombreux thèmes spécifiques aux *softs skills* et à la gestion de projet sont pourtant cruciaux pour de nombreuses entreprises à travers le monde, mais en soit peu traitées par les cours universitaires.

Pour finir, nous avons tracé les fréquences dans les 2 corpus de quelques mots qui nous paraissaient important :

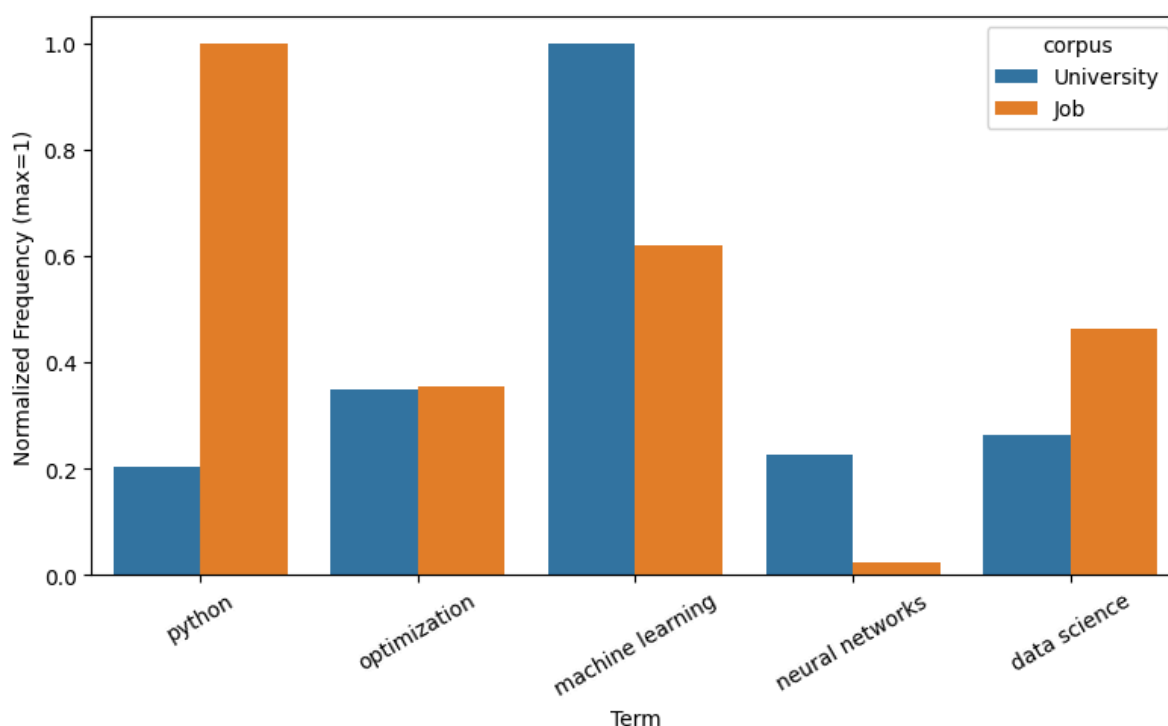


Fig. 12. – Fréquences dans les corpus de quelques mots liés à l'IA

VII.2 LDA : Résultats préliminaires

Dans cette section, nous présentons les résultats préliminaires de l'analyse thématique par LDA (Latent Dirichlet Allocation) appliquée à nos corpus d'universités et d'offres d'emploi, avant l'application de traitements plus poussés et l'utilisation d'une whitelist spécifique. Cette première approche vise à révéler les structures thématiques brutes inhérentes aux données.

VII.2.1 Corpus universitaires

Pour le corpus des universités, les thèmes identifiés sont globalement bien répartis et reflètent le caractère académique des documents. On retrouve des thèmes axés sur des matières ou groupes de matières enseignées, ainsi que d'autres liés au vocabulaire spécifique et aux conditions administratives du monde universitaire.

Voici une ventilation des thèmes identifiés :

- Thème « Conditions Académiques » : Regroupe des termes administratifs et organisationnels des études (ex: unit, prerequisite, team, teach).
- Thème « Mathématiques et Statistiques » : Centré sur les concepts fondamentaux des sciences exactes (ex: math, statistic, mathematic, calculus).
- Thème « Programmation et Systèmes » : Orienté vers l'informatique et l'ingénierie (ex: comp, device, signal, unit, game, optimization).
- Thème « Biologie et Robotique » : Combine des concepts d'ingénierie et de sciences de la vie (ex: robotic, robot, biological, interaction design, computational biology).
- Deux thèmes proches : Traitement de Données / Informatique :
 - Le premier semble plus abstrait et théorique (ex: subject, theorem, compiler, abstraction, reasoning, parallel, java).
 - Le second se concentre davantage sur des aspects spécifiques de l'informatique et du traitement du langage (ex: lecture, semantic, memory, java, signal, optimisation, quantum, classification, natural language).

Cependant, nous avons également identifié quelques thèmes moins distincts, que l'on pourrait qualifier de « fourre-tout », composés de termes plus génériques qui ne forment pas de concepts cohérents sans une whitelist ou un prétraitement plus poussé :

- Exemple de thème « fourre-tout » 1 : (description, point, credit level, subject, modelling, parallel, analyse, module, robot)
- Exemple de thème « fourre-tout » 2 : (module, explain, appropriate, evaluate, able, modelling, protocol, graphic, design)

Ces thèmes « fourre-tout » mettent en évidence la nécessité d'un affinage du prétraitement pour isoler les concepts les plus pertinents.

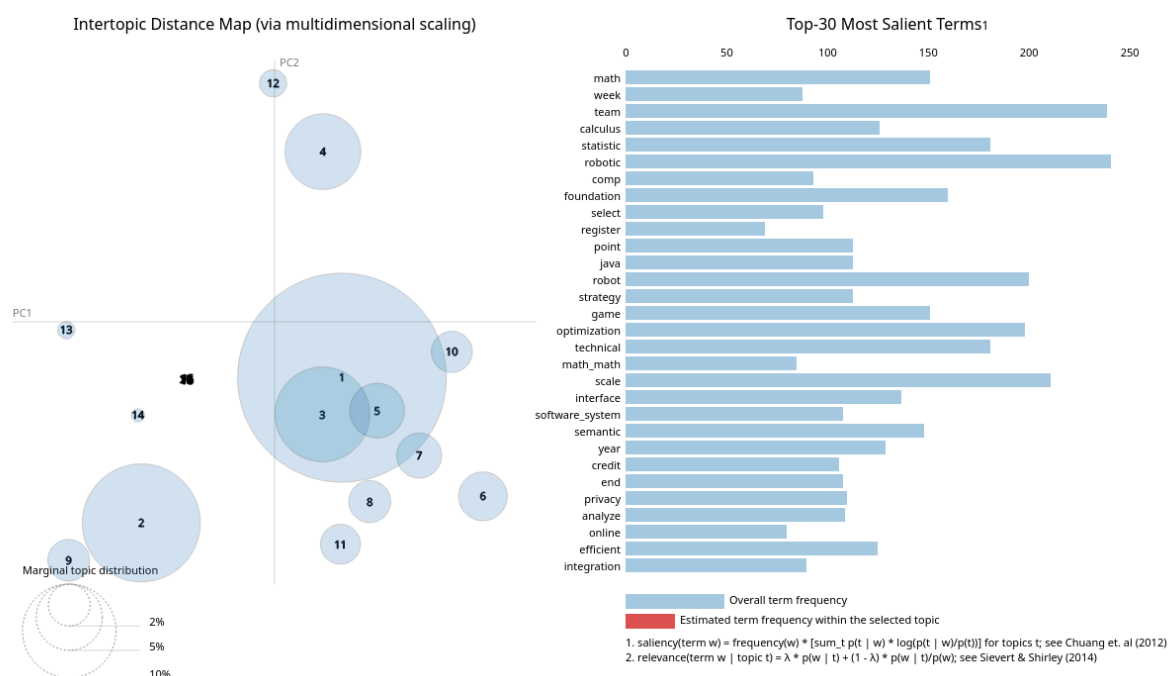


Fig. 13. – Top 30 des mots les plus fréquents par thème pour le corpus Universités (résultats préliminaires).

VII.2.2 Corpus Offres d'emploi

Contrairement aux résultats obtenus pour les universités, l'analyse des offres d'emploi révèle une concentration sur des considérations pratiques et des conditions de travail. On observe une forte occurrence de termes liés aux aspects financiers (salaires, pensions), aux horaires de travail, ainsi qu'aux valeurs d'entreprise telles que la diversité, l'inclusion et le développement professionnel. Un aspect notable est la détection fréquente de groupes de mots entiers par le modèle, soulignant la nature idiomatique et contextuelle de la terminologie des offres d'emploi.

Les catégories générées sont également plus « bruitées » que celles des universités. Cela s'explique logiquement par la nature intrinsèquement plus hétérogène du corpus d'offres d'emploi, qui couvre une multitude de secteurs et de rôles, par rapport au corpus universitaire, principalement axé sur les descriptions de cours.

Les thèmes identifiés incluent :

- Thème « Conditions de Travail et Avantages » : Relatif aux aspects pratiques de l'emploi (ex: schedule day shift, time type subject, benefit program retirement, competitive pay stock bonus, job post detail).
- Thème « Attentes et Responsabilités de l'Employé » : Décrit les qualifications et les tâches (ex: plan facilitate, qualification possess, salary range position, develop sustain, maintain operational excellence).
- Thème « Éducation et Qualifications » : Met en avant les exigences académiques et les compétences nécessaires (ex: demonstrate capacity, reasonable adjustment, selection criterion, tertiary qualification, educational program, sound knowledge).
- Thème « Valeurs d'Entreprise et Inclusion » : Prédominant, ce thème reflète l'accent mis sur la culture d'entreprise et les politiques de diversité (ex: privacy notice privacy page, employer discriminate basis race, pension scheme, value passion discover, invent simplify build, veteran status disability age).

Malgré ces thèmes cohérents, des thèmes « fourre-tout » ou bruités subsistent, similaires à ceux observés dans le corpus universitaire, mais avec une composante spécifique aux offres d'emploi :

- Exemple de thème « fourre-tout » : (canadian, consider asset, rience, pension plan, indigenious, health dental, interview contact). Ces termes sont souvent des résidus de prétraitement ou des expressions moins significatives dans un contexte thématique.

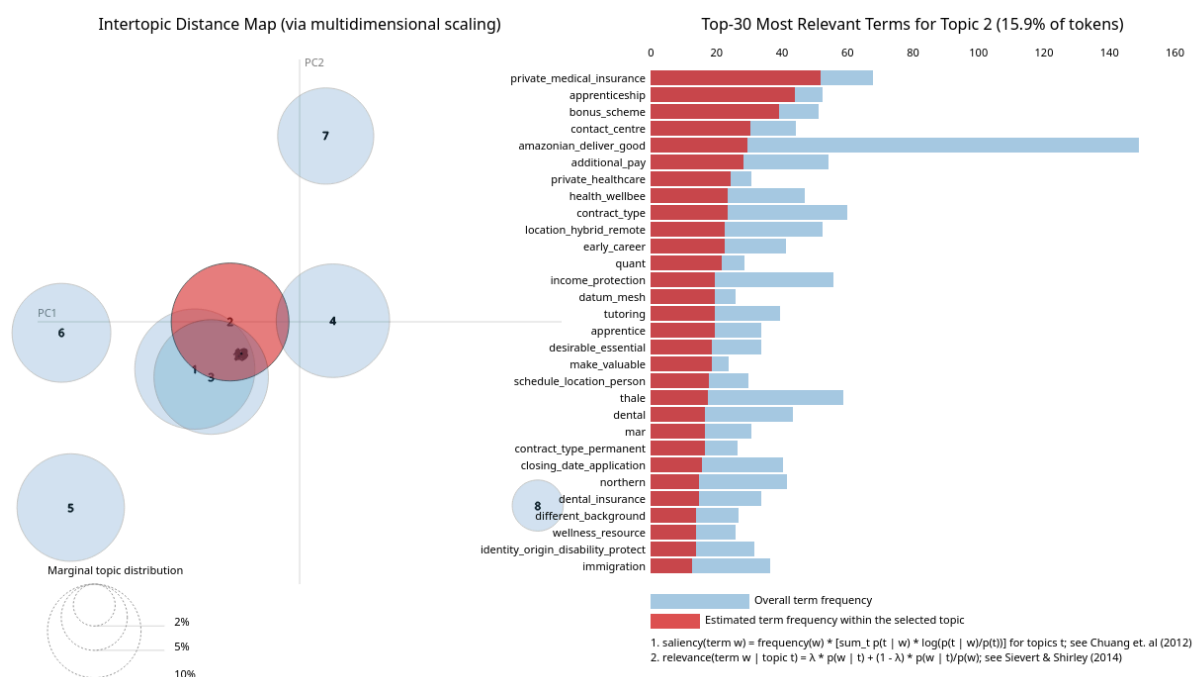


Fig. 14. – Top 30 des mots les plus fréquents par thème pour le corpus Jobs (résultats préliminaires).

VII.2.3 Analyse des distances de Jensen-Shannon

En visualisant les résultats avec la matrice des distances de Jensen-Shannon (figure), nous constatons que les distances sont globalement très proches de 1. Une distance de 1 indique une dissimilitude maximale entre les distributions thématiques des documents, suggérant une très faible similarité. Cela retranscrit la forte divergence thématique entre le corpus des universités et celui des offres d'emploi, mais aussi une cohérence limitée au sein de chaque corpus à ce stade préliminaire. Cette observation confirme la nécessité d'un traitement plus fin pour révéler des structures thématiques plus précises et comparables.

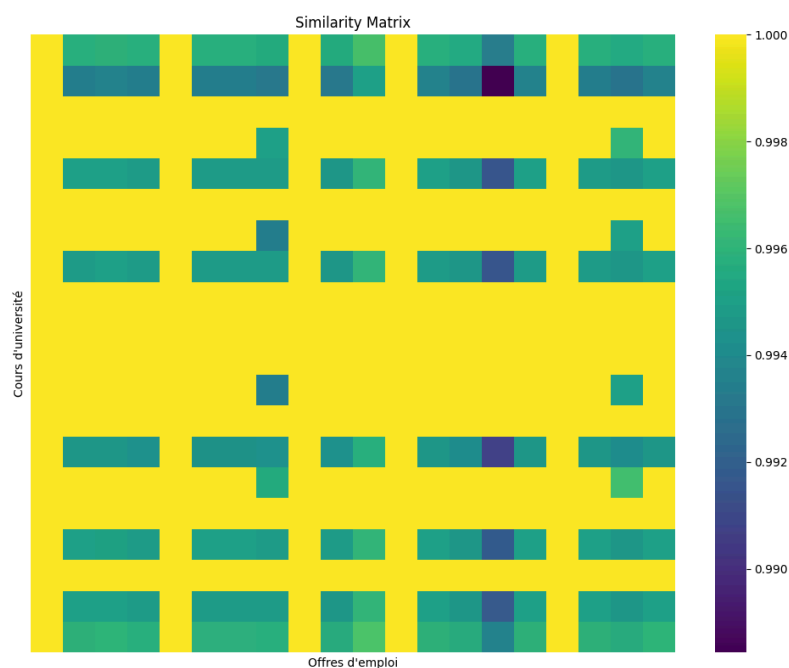


Fig. 15. – Matrice des distances de Jensen-Shannon pour les résultats préliminaires.

VII.3 LDA + Cosinus

Nous avons commencé pour notre modèle de LDA par ne considérer qu'un seul modèle de LDA appliqué à chacun des 2 corpus. Cette approche permet d'utiliser une méthode plus simple pour le calcul de la similarité : la similarité cosinus. Cette méthode permet aussi d'obtenir des thèmes globaux aux 2 corpus de textes qui sont considérés comme des documents d'un même corpus. Cependant, cela implique que la plupart des thèmes sont trop globaux et ne couvrent pas bien les documents : on a notamment des difficultés à avoir des thèmes précis. Comme dans le cas précédent, cela a mené à de nombreux thèmes fouillis considérés comme important par le modèle car contenant beaucoup de mots assez fréquent mais qui ne traduisent pas un thème spécifique.

De ce fait, lors de la comparaison avec la méthode cosinus, les résultats sont encore une fois assez mauvais, avec une faible ressemblance (ici en bleu contrairement à précédemment).

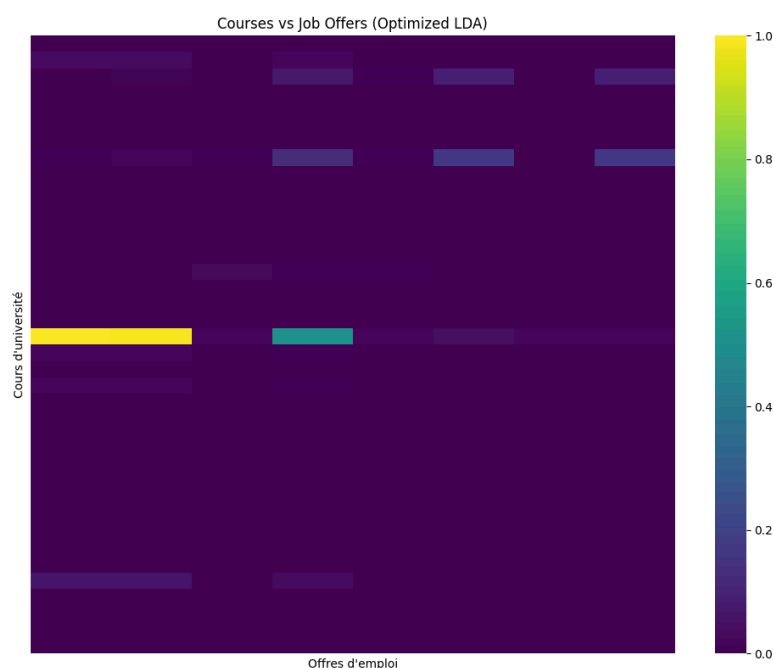


Fig. 16. – Matrice de Jansen Shannon pour nos premiers résultats

VII.4 LDA + Jensen-Shannon

Pour améliorer les performances de notre modèle nous avons donc choisi une nouvelle approche : entraîner deux modèles de LDA différents sur chacun des corpus puis les comparer entre eux. Cependant, en faisant cela, il n'est désormais plus possible d'utiliser la distance cosinus qui n'est pas appropriée pour comparer 2 modèles différents entre eux. On a donc utilisé la distance de Jansen, qui va comparer les distributions de probabilités intrinsèques aux 2 modèles.

VII.4.1 Corpus des cours universitaire

Sur le corpus des cours universitaires, en ajustant la liste de *stopwords* et en utilisant la *whitelist*, on obtient une description fine des documents avec de nombreux thèmes précis et variés. Ces thèmes montre que notre implémentation a permis de surmonter les problèmes initiaux : des thèmes trop globaux, redondants. La Fig. 17 montre que l'on est parvenu à obtenir de nombreux thèmes, divers et précis. Ces thèmes incluent :

- un thème « Statistique » : credit, statistic, reasoning stat analyse
- un thème « Analyse de donnée » : comprenant pass, computing, big data

- 3 thèmes « Programmation » : comprenant des mots comme java, computing, memory...
- un thème « Applications » : décrivant des thèmes variés comme la santé ou bien la bio-informatique.

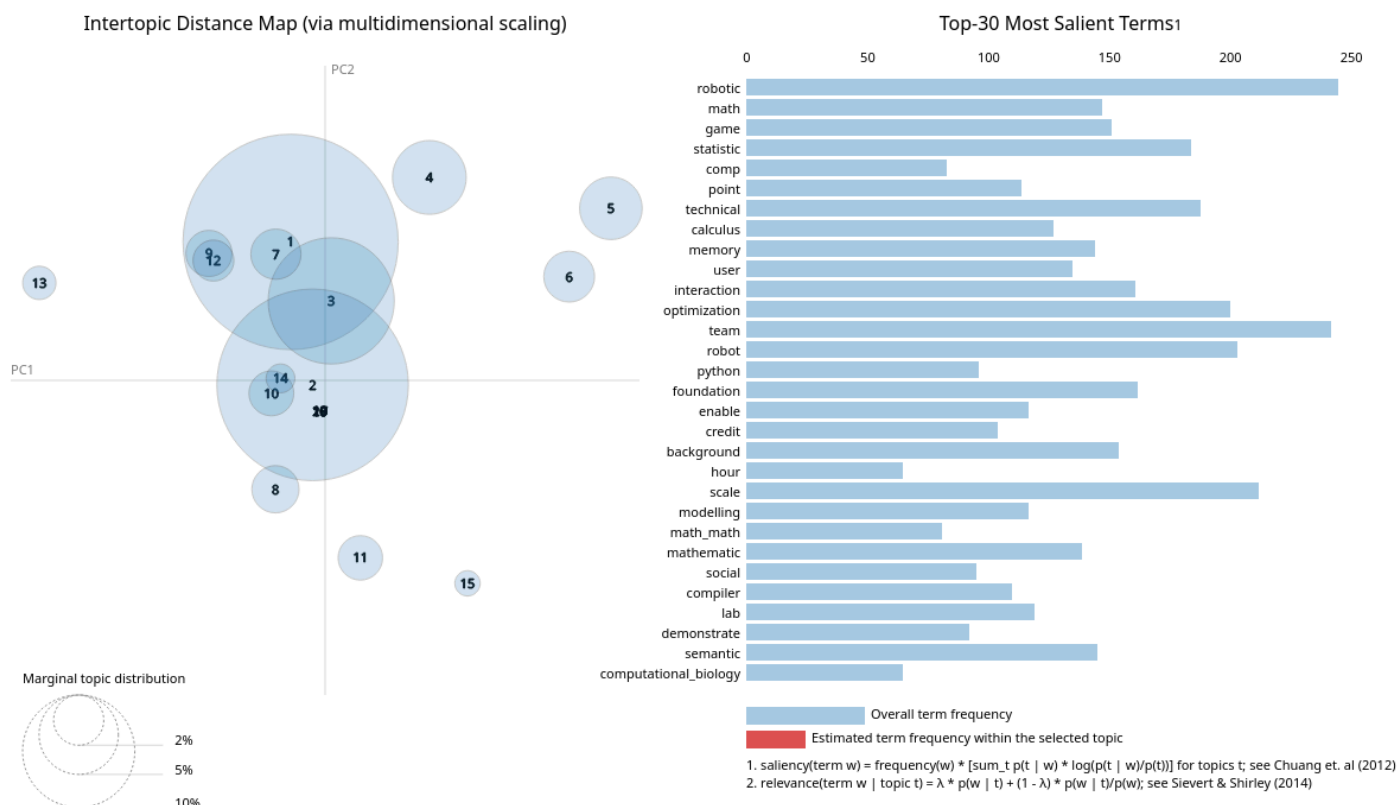


Fig. 17. – Résultats finaux de la LDA pour le corpus université

VII.4.2 Corpus des offres d'emploi

Après avoir supprimé les thèmes autres, et s'être concentrés principalement sur les compétences techniques on remarque cette fois que si les mots qui ressortent sont assez significatifs, cela donne des thèmes qui se recoupent tous, orientés autour de l'analyse de donnée, la science des données et la cybersécurité.

Cet ensemble forme donc la majorité des attentes des recruteurs, et l'incapacité du modèle à dissocier d'avantage les thèmes des documents peut signifier que les attentes des recruteurs se recoupent beaucoup sur un petit nombre d'attentes très importantes, en plus de s'axer en grande partie sur une composante plus orientée travail de groupe ou conditions de travail

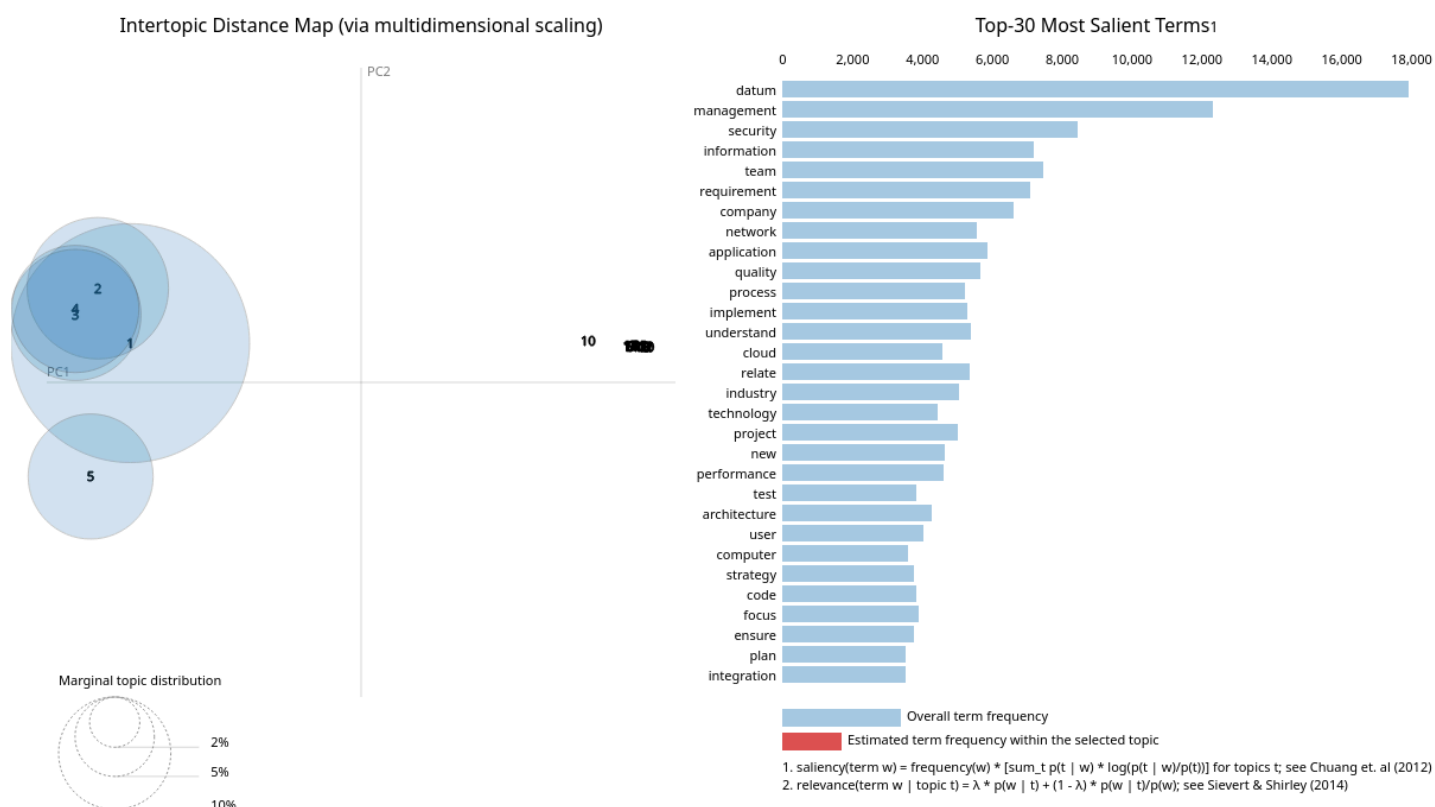


Fig. 18. – Résultats finaux de la LDA pour le corpus offres d'emploi

Finalement, ces résultats nous permettent d'obtenir une nouvelle matrice de Jansen qui est bien meilleure puisqu'elle montre une plus grande variabilité mais aussi une moyenne plus basse. Cela révèle que les différents thèmes trouvés sont plus proches les uns des autres :

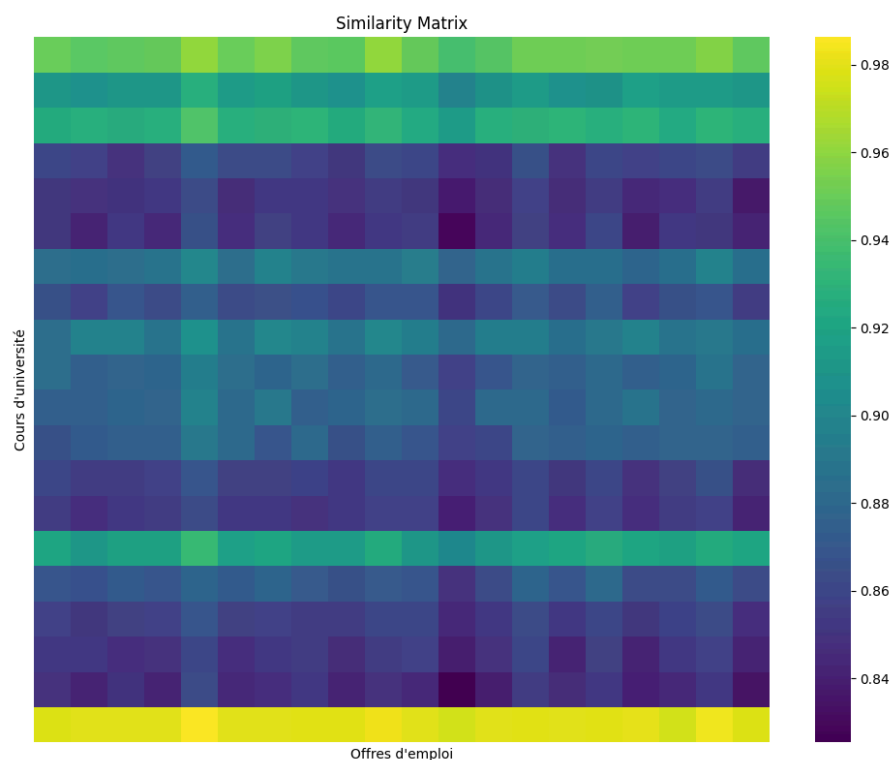


Fig. 19. – Matrice de Jansen

VII.5 BERT + Cosinus

La similarité cosinus appliquée aux embeddings de BERT donne un résultat très satisfaisant, mettant en avant des différences notables entre universités, que nous pouvons analyser assez simplement. En fait, l'intérêt d'avoir un calcul de similarité aussi précis (chaque syllabus d'université est comparé aux offres d'emploi des 8 pays distinctement) est d'exploiter plus rapidement les différences.

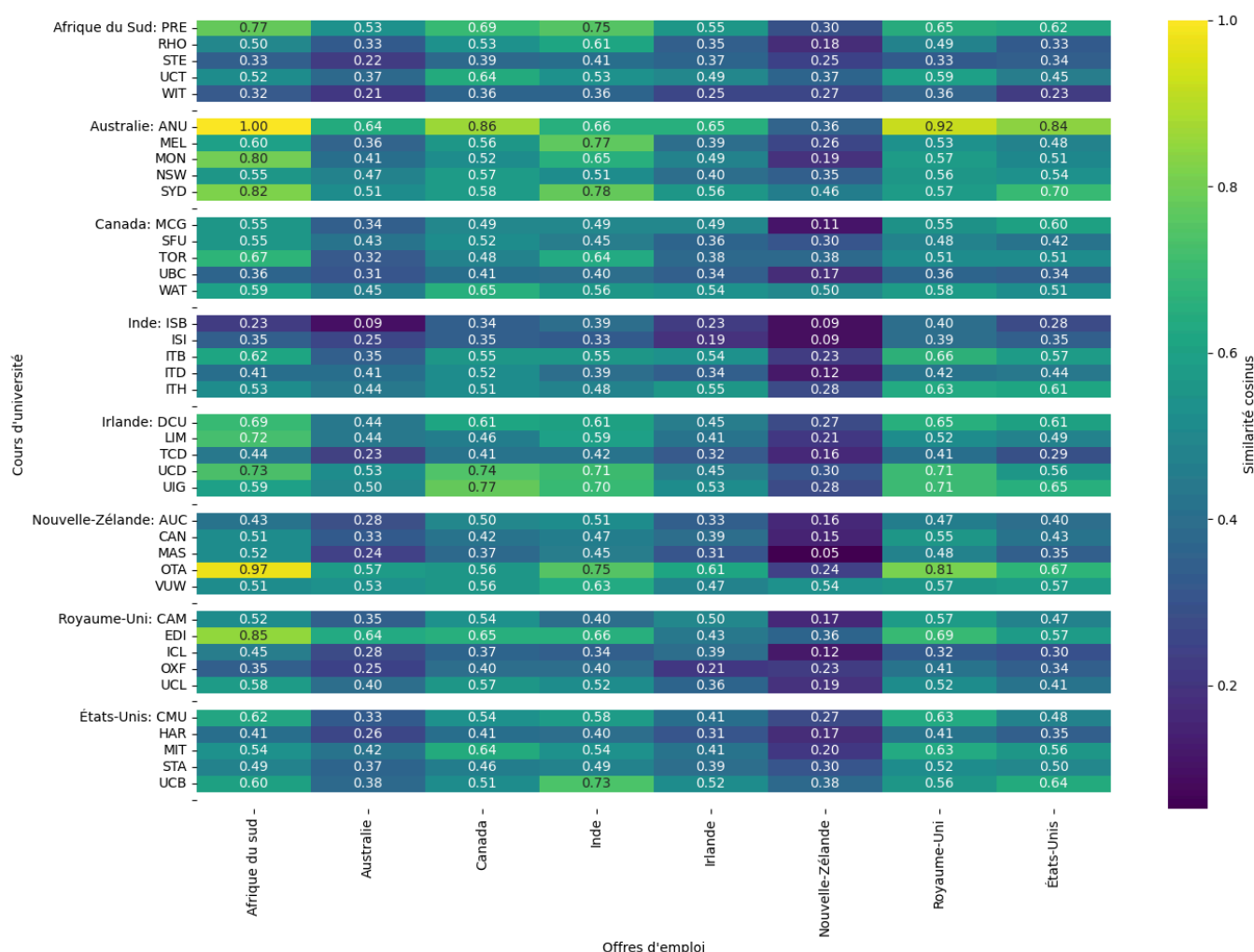


Fig. 20. – Matrice de similarité cosinus avec BERT

Nous remarquons une colonne particulière : la colonne de la Nouvelle-Zélande a une similarité plus faible que celle des autres pays, ce qui peut s'expliquer par le Chapitre V.1.2.1.2, révélant le manque d'offres d'emploi lors du *scraping* de ce pays.

VII.6 Tests de comparaison à différentes échelles avec BERT

Il nous semble que les résultats obtenus avec BERT sont les plus pertinents. Nous avons décidé de poursuivre notre étude en analysant la matrice de similarité cosinus obtenue avec ce modèle (Fig. 20). Pour cela, nous avons réalisé plusieurs tests en comparant les cours et les offres d'emploi à différentes échelles :

- Cours d'un pays vs. Offres d'emploi du même pays → **8 tests**
- Cours d'un pays vs. Offres d'emploi de tous les pays → **8 tests**
- Cours d'une université vs. Offres d'emploi du même pays → $5 \times 8 = 40$ **tests**

- Cours d'une université vs. Offres d'emploi de tous les pays $\rightarrow 5 \times 8 = 40$ tests
- Cours de toutes les universités vs. Offres d'emploi d'un pays $\rightarrow 8$ tests
- Cours de toutes les universités vs. Offres d'emploi de tous les pays $\rightarrow 1$ test

Ces **105 tests** permettent d'identifier les écarts potentiels entre la formation académique et les attentes du marché à différents niveaux de granularité.

VII.6.1 Cours d'un pays vs. Offres d'emploi d'un même pays

Question : Les universités d'un pays proposent-elles des cours adaptés au marché de l'emploi de leur propre pays ?

Un réarrangement des colonnes de la matrice de similarité mentionnée sous la référence Fig. 20, cette fois classée par pays, permet d'obtenir une nouvelle version. Cette opération facilite l'analyse des correspondances entre les universités et les offres d'emploi en regroupant les établissements par pays d'origine. Une fois cette organisation effectuée, il suffit de calculer la moyenne des similarités associées à chaque université d'un même pays. Cela permet de déterminer si, en moyenne, les universités d'un pays particulier affichent une forte similarité avec les offres d'emploi provenant de ce même pays. Pour ce faire, on observe les valeurs situées sur la diagonale de la matrice, qui correspondent aux cas où le pays d'origine des universités et celui des offres d'emploi sont identiques.

Pays	Moyenne des cours et offres d'emploi du pays
Afrique du Sud	0.489
Australie	0.477
Canada	0.510
Inde	0.427
Irlande	0.432
Nouvelle-Zélande	0.228
Royaume-Uni	0.5051
États-Unis	0.5052

Tableau 3. – Résultat des universités et des emplois d'un même pays

On remarque, sur le Tableau 3, que le Canada possède la valeur la plus élevée suivie de très près par les États-Unis et le Royaume-Uni. Ce classement est plutôt logique, les pays du Nord sont économiquement plus riches et donc moins susceptibles à la fuite de cerveaux. Il n'est pas surprenant de retrouver la Nouvelle-Zélande en dernier, à cause du *scraping* qui ne nous a permis que de récupérer 223 emplois, cette valeur est donc erronée et ne permet pas d'être interprétée.

VII.6.2 Cours d'un pays vs. Offres d'emploi de tous les pays

Question : Les universités d'un pays proposent-elles des cours cohérents au marché de l'emploi national et international ?

Le Tableau 4 montre que les universités australiennes ont le cursus le plus aligné, en moyenne, avec les offres d'emploi du monde entier. Cela pourrait indiquer une orientation internationale de leurs formations, visant à préparer des diplômés compétitifs à l'échelle globale. À l'inverse,

l'Inde affiche la moyenne la plus basse, ce qui pourrait suggérer un besoin d'adaptation ou de diversification des contenus proposés.

Université de...	Moyenne des cours d'un pays et offres d'emploi de tous les pays
Afrique du Sud	0.428
Australie	0.574
Canada	0.452
Inde	0.387
Irlande	0.508
Nouvelle-Zélande	0.467
Royaume-Uni	0.427
États-Unis	0.460

Tableau 4. – Résultat des universités d'un pays et des emplois de tous les pays

Le Tableau 4 montre que les universités australiennes ont le cursus le plus aligné, en moyenne, avec les offres d'emploi du monde entier. Cela pourrait indiquer une orientation internationale de leurs formations, visant à préparer des diplômés compétitifs à l'échelle globale. À l'inverse, l'Inde affiche la moyenne la plus basse, ce qui pourrait suggérer un besoin d'adaptation ou de diversification des contenus proposés. De plus, il est surprenant de voir que la Nouvelle-Zélande n'est pas dernière sur ce classement, cela montre en effet que la Nouvelle-Zélande reste compétitive à l'international malgré le manque d'offres d'emploi obtenu lors du *scraping*.

VII.6.3 Cours de toutes les universités vs. Offres d'emploi d'un pays

Question : Les offres d'emploi d'un pays sont-ils en concordance avec les cours universitaires mondiaux ?

Ce test inverse la perspective : on cherche ici à savoir si les besoins exprimés par un marché du travail national correspondent aux formations disponibles à l'échelle mondiale. Pour chaque pays, on calcule donc la moyenne des similarités entre ses offres d'emploi et l'ensemble des cours d'université, tous pays confondus.

Pays	Moyenne des offres d'emploi d'un pays et de toutes les universités
Afrique du Sud	0.565
Australie	0.381
Canada	0.526
Inde	0.540
Irlande	0.417
Nouvelle-Zélande	0.250
Royaume-Uni	0.542
États-Unis	0.483

Tableau 5. – Résultat des emplois d'un pays et de toutes les universités

D'après le Tableau 5, le marché de l'emploi sud-africain est celui qui correspond le mieux à l'offre universitaire globale, suivi par le Royaume-Uni, l'Inde et le Canada. L'Australie, en revanche, affiche une inadéquation marquée, ce qui peut indiquer des attentes spécifiques non couvertes par la majorité des cursus.

VII.6.4 Cours d'une université vs. Offres d'emploi du même pays

Question : Certaines universités se distinguent-elles par une forte correspondance avec leur marché de l'emploi national ?

Pour ces tests, nous nous référons à la Fig. 20. Pour chaque pays, on obtient les universités le mieux et le moins bien classées dans le Tableau 6,

Pays	Université (similarité cosinus)
Afrique du Sud	PRE (0.77)
	WIT (0.32)
Australie	ANU (0.64)
	MEL (0.36)
Canada	WAT (0.65)
	UBC (0.41)
Inde	ITB (0.55)
	ISI (0.33)
Irlande	UIG (0.53)
	TCD (0.32)
Nouvelle-Zélande	VUW (0.54)
	MAS (0.05)
Royaume-Uni	EDI (0.69)
	ICL (0.32)
États-Unis	UCB (0.64)
	HAR (0.35)

Tableau 6. – Première et dernière université pour chaque pays

VII.6.5 Cours d'une université vs. Offres d'emploi de tous les pays

Question : Certaines universités sont-elles particulièrement en phase avec le marché de l'emploi global ?

Il s'agit d'analyser établissement par établissement la capacité à former des diplômés compétitifs à l'échelle mondiale. Sur la matrice de similarité de la Fig. 20, nous trouvons que les universités ANU et OTA performant le mieux sur le marché de l'emploi sud-africain. En revanche l'université néo-zélandaise de MAS à une cohésion extrêmement basse dans le marché de l'emploi de son propre pays. Il s'agit de valeur aberrante dû au manque d'offres d'emploi récupérées et donc inexploitable.

Un autre point remarquable vient de la ligne de l'université d'Australian National University (ANU). Il possède non seulement le maximum de similarité dans toute la matrice (maximum

qui est lié à l'Afrique du Sud), mais aussi qu'elle a une similarité meilleure dans presque tous les pays comparé aux autres universités.

VII.6.6 Cours de toute les universités vs. Offres d'emploi de tous les pays

Question : Les universités dans leur ensemble sont-elles en adéquation avec le marché de l'emploi mondial ?

En moyenne, la similarité observée entre l'ensemble des cursus universitaires et les offres d'emploi de tous les pays est de 0.463, ce qui indique un alignement modéré. Cela suggère qu'il existe encore une marge de progression pour rapprocher les contenus pédagogiques des besoins concrets du monde professionnel à l'échelle internationale.

VIII Conclusion

Cette étude s'est attachée à évaluer l'alignement entre les formations universitaires internationales et les besoins concrets du marché de l'emploi, en s'appuyant sur des outils avancés de traitement automatique du langage (notamment BERT et LDA) et des métriques de similarité sémantique.

Les résultats obtenus montrent des tendances claires mais nuancées :

- À l'échelle nationale, certains pays comme le Canada, le Royaume-Uni et les États-Unis affichent des correspondances élevées entre leurs cursus et leurs offres d'emploi nationales, traduisant un bon alignement institutionnel.
- En revanche, des pays comme la Nouvelle-Zélande ou l'Australie présentent des résultats plus contrastés, révélant soit des lacunes dans la couverture des compétences recherchées, soit des biais liés aux limitations du *scraping* ou des corpus analysés (notamment en ce qui concerne la Nouvelle-Zélande).
- À l'échelle internationale, les universités australiennes se démarquent par leur adéquation aux besoins mondiaux, tandis que les universités indiennes apparaissent moins alignées. Cela pourrait refléter à la fois des choix stratégiques locaux et des contraintes économiques ou structurelles spécifiques.

En moyenne, la similarité globale mesurée entre l'ensemble des cursus universitaires et les offres d'emploi mondiales atteint 0,463, ce qui correspond à un alignement modéré. Cela souligne qu'il reste des marges d'amélioration pour rapprocher davantage l'enseignement supérieur des réalités économiques, notamment en actualisant régulièrement les contenus pédagogiques et en intégrant plus rapidement les innovations et technologies émergentes.

D'un point de vue méthodologique, ce projet illustre l'apport essentiel des modèles sémantiques profonds pour dépasser les analyses classiques basées sur des mots-clés et explorer les correspondances réelles entre compétences enseignées et requises. Les visualisations, les analyses de blocs par pays et les moyennes de similarité ont permis de dégager des *insights* précis, tout en mettant en lumière certaines limites techniques, notamment la qualité et la quantité des données récupérées selon les régions.

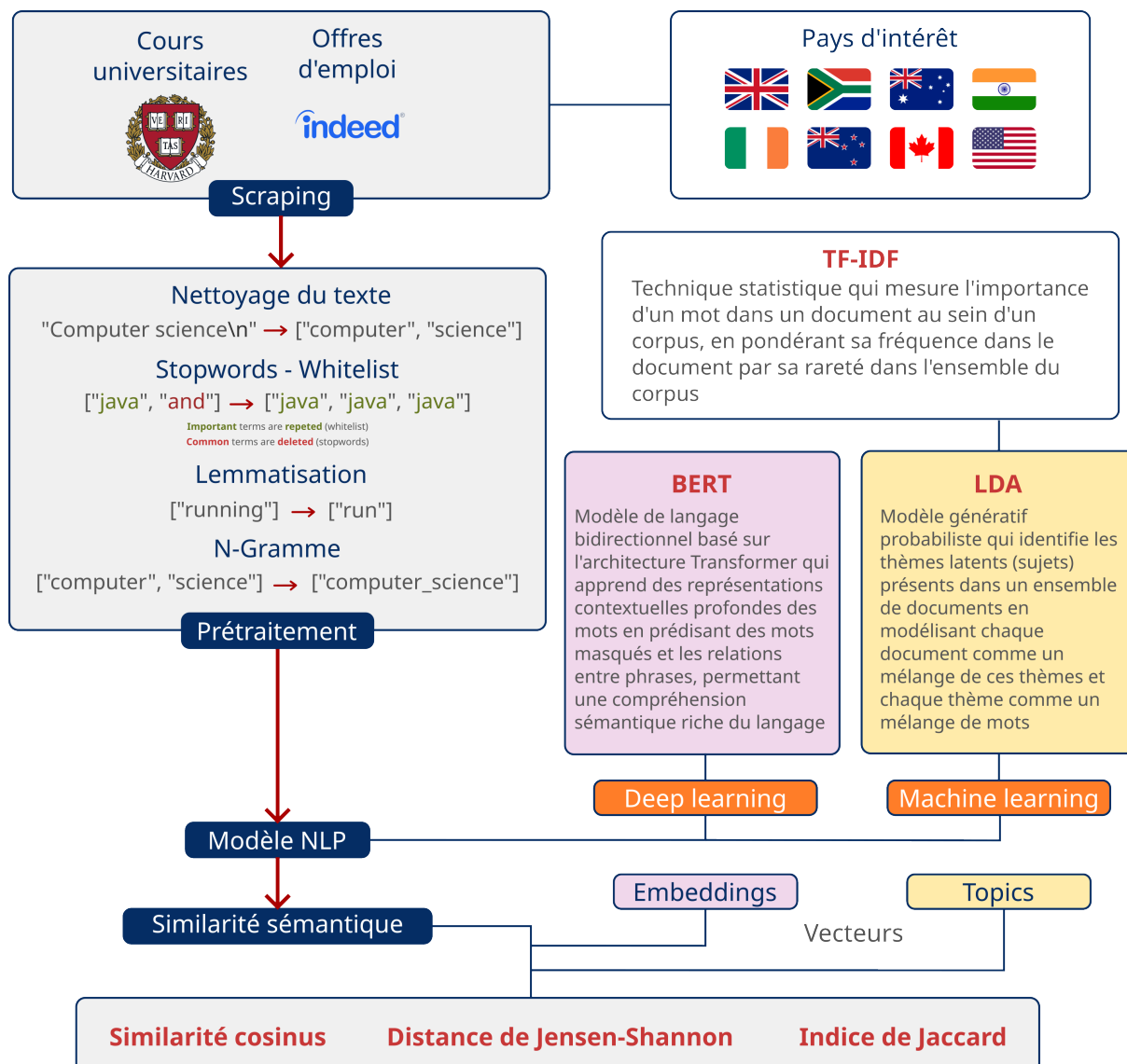
En conclusion, cette étude constitue une avancée significative vers une cartographie fine des écarts formation-emploi dans le domaine de l'intelligence artificielle, offrant aux institutions académiques et aux décideurs un point d'appui pour réformer les programmes, renforcer l'employabilité des diplômés et mieux répondre aux défis d'un marché globalisé et en mutation constante.

IX Annexes

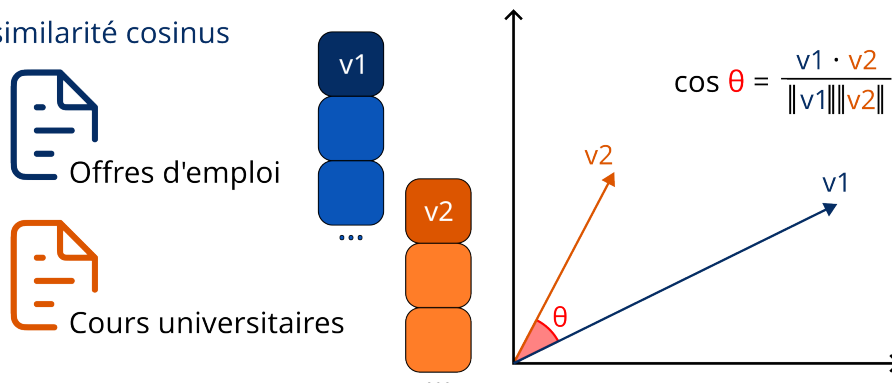
IX.1 Annexe 1 : Liste des universités étudiées

Pays	Université
États-Unis d'Amérique	Massachusetts Institute of Technology (MIT)
	Stanford University (STA)
	Carnegie Mellon University (CMU)
	Harvard University (HAR)
	University of California, Berkeley (UCB)
Royaume-Uni	University of Oxford (OXF)
	University of Cambridge (CAM)
	Imperial College London (ICL)
	University College London (UCL)
	University of Edinburgh (EDI)
Canada	University of Toronto (TOR)
	University of British Columbia (UBC)
	McGill University (MCG)
	University of Waterloo (WAT)
	Simon Fraser University (SFU)
Australie	University of Melbourne (MEL)
	Australian National University (ANU)
	University of Sydney (SYD)
	University of New South Wales (NSW)
	Monash University (MON)
Inde	Indian Institute of Technology Bombay (ITB)
	Indian Institute of Science Bangalore (ISB)
	Indian Institute of Technology Delhi (ITD)
	Indian Statistical Institute (ISI)
	IIT Hyderabad (ITH)
Afrique du Sud	University of Cape Town (UCT)
	University of the Witwatersrand (WIT)
	Stellenbosch University (STE)
	University of Pretoria (PRE)
	Rhodes University (RHO)
Irlande	Trinity College Dublin (TCD)
	University College Dublin (UCD)
	National University of Ireland Galway (UIG)
	Dublin City University (DCU)
	University of Limerick (LIM)
Nouvelle-Zélande	University of Auckland (AUC)
	Victoria University of Wellington (VUW)
	University of Canterbury (CAN)
	Massey University (MAS)
	University of Otago (OTA)

IX.2 Annexe 3 : Schéma récapitulatif de notre pipeline NLP



Exemple : similarité cosinus



IX.3 Annexe 3 : *Whitelist*

- **Langages de programmation** : « python », « java », « c++ », « javascript », « ruby », « swift », « rust », « php », « typescript », « kotlin », « scala », « r », « dart », « perl », « lua », « haskell », « julia », « matlab », « sql », « c# », “c”, « objective-c », « groovy », « shell », « powershell », « assembly », « cobol », « fortran », « ada », « lisp », « prolog », « erlang », « elixir », « clojure », « f# », « ocaml », « vb.net », “html”, “css”,
- **Outils mathématiques** : « linear algebra », « differential calculus », « statistics », « probability », « optimization », « graph theory », « numerical analysis », « mathematical logic », « set theory », « topology », « number theory », « differential equations », « complex analysis », « discrete mathematics », « fourier analysis », « game theory », « operations research », « combinatorics »,
- **Intelligence artificielle** : « machine learning », « neural networks », « deep learning », « natural language processing », « computer vision », « genetic algorithms », « robotics », « expert systems », « data mining », « pattern recognition », « automated reasoning », « knowledge representation », « reinforcement learning », « supervised learning », « unsupervised learning », « semi-supervised learning », « transfer learning », « ensemble methods », « dimensionality reduction », « feature engineering »,
- **Autres outils et technologies** : « git », « docker », « kubernetes », « tensorflow », « pytorch », « scikit-learn », « pandas », « numpy », « scipy », « apache spark », « hadoop », « tableau », « power bi », « excel », « jupyter », « ansible », « chef », « puppet », « jenkins », « travis ci », « circleci », « github », « gitlab », « selenium », « postgresql », « mysql », « mongodb », « redis », « elasticsearch », « kafka », « rabbitmq », « flask », « django », « spring boot », « node.js », « express.js », “bootstrap”, “symfony”,
- **Concepts et méthodologies** : « agile », « devops », « big data », « cloud computing », « cybersecurity », « blockchain », « internet of things », “iot”, « augmented reality », « virtual reality », « quantum computing », « microservices », « continuous integration », « continuous deployment », « mlops », « data science », « data engineering », « software engineering », « system architecture », « network security », « cryptography », « human-computer interaction », « user experience design », « software testing », « quality assurance », « project management », « scrum », « kanban »

IX.4 Annexe 4 : Diagramme de Gantt

